University of Plymouth

School of Engineering, Computing, and Mathematics

PRCO304

Final Stage Computing Project

2019/2020

Swell Surf Check

Emily Potter

10521234

BSc (Hons) Computer Science

Acknowledgments

I would like to express my gratitude to Thomas Wennekers for his feedback and guidance throughout this project which ultimately led to a stronger end product.

I would also like to extend my thanks to my family and friends who have supported me through the project and helped out wherever they could, including taking part in the usability sessions to improve Swell Surf Check.

Abstract

This report will describe a software development project for a surf forecast and information Web and Android Application. There are plans to release the Android Application to the Play Store in the future so users all over the world can access it and can be accessed from any web browser by simply navigating to the given URL.

The report begins with a look into the background of surf forecast websites and why they are so popular and then goes into the aims and objectives of this project. Next, it investigates the technologies used, including MongoDB, Node.js, Angular and Serverless and each choice is justified by research.

The development process used for the project was agile which allows for an incremental approach to the project, with freedom to change schedule or ideas if needed. During each sprint, a new piece of functionality was added. The legal, professional, and ethical issues are described, and an analysis of the project management approach is done. The architecture of the project is explored with diagrams depicting the choices made and the user and functional requirements are reviewed.

The report then highlights each stage of the development process where key accomplishments, challenges and deliverables were examined, before an overview of the user testing and the automated testing took place.

A project post-mortem was then presented, where included requirements were evaluated along with what could have gone better and future enhancements to improve the final product.

Contents

Contents 3
Statement of Word Count 6
Code Submission 6
List of Figures and Tables 7
1. Introduction 8
1.1 Background 8
1.2 Aims and Objectives 8
1.3 Deliverables 9
1.4 Report Overview 9
2. Literature Review 9
2.1 Technological Justification 9
2.1.1 Angular 9
2.1.2 MongoDB 10
2.1.3 AWS Lambda 10
2.1.4 Ionic 10
2.2 Chatbot 11
2.3 Android HCI 11
2.4 Web HCI 12
2.5 Existing Product Analysis 12
2.5.1 Magic Seaweed 12
3. Method of Approach 13
4. Legal, Social, Ethical and Professional Issues 14
5. Project Management 15
5.1 Supervisor Meetings 15
5.2 Version Control and DevOps 16
5.2 User Management 17
5.3 Time Management 17
6. Requirements 17
6.1 Functional Requirements 17
6.2 User Requirements 19
6.3 Non-Functional Requirements 20
7. Architecture 20
7.1 Architecture Overview 21

7.2 Database Schema 22
7.3 UML Diagrams 22
8. Design 23
8.1 Initial User Interface Design 23
8.2 Prototype Development 23
8.3 User Testing 24
8.3.1 Web Application 24
8.3.2 Android Application 26
9. Project Stages 28
9.1 Stage 1 28
9.1.1 Summary 28
9.2 Stage 2 28
9.2.1 Summary 28
9.2.2 Deliverables 29
9.2.3 Key Challenges and Accomplishments 29
9.3 Stage 3 29
9.2.1 Summary 29
9.2.2 Deliverables 29
9.4 Stage 4 29
9.4.1 Summary 29
9.4.2 Deliverables 30
9.4.3 Key Challenges and Accomplishments 30
9.5 Stage 5 30
9.5.1 Summary 30
9.2.2 Deliverables 31
9.2.3 Key Challenges and Accomplishments 31
9.6 Stage 6 31
9.6.1 Summary 31
9.6.2 Deliverables 32
9.6.3 Key Challenges and Accomplishments 33
9.7 Stage 7 33
9.7.1 Summary 33
9.7.2 Deliverables 34
9.7.3 Key Challenges and Accomplishments 34
10. Testing 34

11. End Project Report 35
12. Project Post-Mortem 35
12.1 What Went Well? 35
12.2 What Didn't Go Well? 36
12.3 What Would Be Performed Differently? 37
12.4 Future Enhancements 37
13. Conclusions 37
14. References 39
15. Appendices 42
15.1 User Guide 42
15.2 Project Management 52
15.2.1 Trello Boards 52
15.2.2 Requirements 56
15.2.3 Stage Plans 58
15.2.4 Sprint Reviews 62
15.3 UML Diagrams 64
15.4 Testing 65
15.5 Wire Frame Diagrams 69
15.6 Questionnaire Results 71
15.7 API Endpoints 80
15.8 AWS Lambda Code 81
15.9 Dialogflow Fulfilment Code 93

Statement of Word Count

Word count: 9,991.

Code Submission

Web: https://github.com/emilypotter/PRCO304-Web

Mobile: https://github.com/emilypotter/PRCO304-Mobile

List of Figures and Tables

Figure 1 - Magic Seaweed mobile apps (Magic Seaweed, 2020) 12
Figure 2 – Sprint (Visual Paradigm, n.d.) 13
Figure 3 - An initial idea of the tasks to be done 14
Figure 4 - Powered by Google logo (Google, 2020) 15
Figure 5 - Heroku auto build and deploy from GitHub, including version number. ... 16
Figure 6 - High level architecture diagram 21
Figure 7 - Database schema 22
Figure 8 - Activity diagram: add a spot to favourites 22
Figure 9 - Use case: add a spot to favourites 23
Figure 10 - Questionnaire results 25
Figure 11 - Improved home screen 25
Figure 12 - Improved spot detail page 26
Figure 13 - Improved questionnaire results 26
Figure 14 - Questionnaire results 27
Figure 15 - Improved mobile home screen 27
Figure 16 - Improved questionnaire results 28
Figure 17 - Automated unit tests running with Karma 35
Figure 18 - Attempting to call the onMessage function on the connected web socket API but getting a ‘forbidden’ message along with the connectionId. 37

Table 1 - Project objectives 8
Table 2 - Android guidelines on visual design and user interaction (Android, 2020) 11
Table 3 - Functional requirements 17
Table 4 - User requirements 19
Table 5 - Non-functional requirements 20

1. Introduction

Surfing is one of the most popular water sports in the world. “The latest study suggests that today there are 35 million surfers who practice their wave-riding activities in at least 162 countries” (Lipchain, 2018). With the numbers growing exponentially and lives getting more hectic in the modern world, surfers are looking for anything that can save them some time. Getting up early to beat the crowds can pay off sometimes but most of the time they will get to the beach to find no waves. Therefore, surfers around the world are now using the internet to find out the conditions in the comfort of their own homes without having to waste a trip to the beach, saving them precious time. However, with very few options on where to find this information, surfers are desperate for a definitive source for information about the conditions.

With tourism in the South West thriving, it is not just surfers looking for information. “The South West represents the highest proportion of all English regions for domestic overnight trips, 19% of total trips, and 23% of spend, in 2015 – 19.7 million trips and £4.4 billion in spend” (Visit Britian, 2015). Tourists are a major income for the South West and giving them information to better their stay is crucial.

1.1 Background

Swell Surf Check provides surfers and non-surfers alike with information on surf spots and points of interest in the areas surrounding each spot. The Web Application also has an Artificial Intelligence chat bot to help users find the conditions at their chosen spot.

A user can login or register through either the Web Application or the Android Application in order to save their favourite spots for easy access. Spots can either be searched by per region or, if the user knows the exact spot they need, they can use the autocomplete search. Once selected, there is a detailed page on each spot where the user can browse through a gallery of pictures as well as find points of interest nearby and check the current and future surf conditions.

With Swell Surf Check being available on both desktop and mobile, the user can find the information they need either at home or on the go so the potential user base is expanded beyond one platform.

1.2 Aims and Objectives

The main objectives for Swell Surf Check are described below for each platform.

Table 1 - Project objectives

Objective	Platform
Allow user to search for a surf spot with autocomplete.	Web
Allow user to browse potential surf spots.	Web & Android
Provide user account creation and login to accommodate users saving their favourite spots.	Web & Android
Have a detailed page for each spot providing information on the current and future surf conditions.	Web & Android

Incorporate Google Maps to show nearby points of interest.	Web
Incorporate an AI chat bot that users can “talk to” to get information on current conditions.	Web

1.3 Deliverables

The final deliverables of the project will consist of two applications: A Web Application and an Android Application. Both of these should have successfully fulfilled the objectives listed in section 1.2.

1.4 Report Overview

This report will depict how Swell Surf Check was planned, managed and implemented.

Firstly, the report will provide a literature review, focusing on the main technologies used through development including research into why they have been used and why each was considered the best option for this project. Next, it will go over existing products similar to Swell Surf Check and the issues apparent with them. Following this, the report will detail the method of approach to the project and go over some of the legal, social, ethical and professional issues that must be considered during development. Next, it will highlight how the project was managed and the architectural design of the system. The report will then focus on how the user interface was designed, followed finally by a break down of each stage of development. Conclusions will then be drawn, accompanied by a reference list and appendix.

2. Literature Review

2.1 Technological Justification

The section will define and justify some of the technologies used in Swell Surf Check. It will go into the research done and why it was the best choice for the project.

2.1.1 Angular

“Angular is an app-design framework and development platform for creating efficient and sophisticated single-page apps” (Angular, 2020). It uses Typescript - a superset of JavaScript that allows optional typing to the language. It also uses HTML and many options for styling including CSS and SCSS.

Angular was chosen over other web development frameworks firstly because of the very detailed documentation, making the learning curve for someone who is relatively new to web development much less steep. “Angular boasts detailed documentation where developers can find all necessary information without asking their colleagues. As a result, developers can quickly come up with technical solutions and resolve emerging issues” (Harkushko, n.d.). Angular is also considered very trustworthy because it has the support of Google, who announced in 2017 Angular has their long-term support, making the framework very reliable.

The easily understandable architecture also made Angular a great choice for the project. The app is made up of components which work completely independently from each other, making testing much more straightforward.

2.1.2 MongoDB

"MongoDB is a document database with the scalability and flexibility that you want with the querying and indexing that you need" (MongoDB, n.d.). It is a NoSQL database meaning it uses documents as opposed to tables. These documents are stored in collections instead of the columns and rows of a more traditional relational database.

MongoDB was the choice for Swell Surf Check because of its ease of use and its speed and efficiency. Because queries are based on collections and documents instead of a schema as in a relational database, results are returned much more quickly resulting in a better user experience.

2.1.3 AWS Lambda

Amazon Web Services (AWS) provide serverless architecture in the form of AWS Lambdas. Each Lambda is a standalone piece of code that serves a purpose, for example, getting data from a database.

Some of the main benefits of using this over a hosted server is that there are no servers to manage and AWS Lambda automatically scales the application, so the Lambda is only run when it is triggered. This is much more cost effective than having a constantly running server that can overloaded if too many requests are made. AWS Lambdas are also considered to have very consistent performance. "With AWS Lambda, you can optimize your code execution time by choosing the right memory size for your function. You can also enable Provisioned Concurrency to keep your functions initialized and hyper-ready to respond within double digit milliseconds" (Amazon, n.d.).

The functions themselves can be written in several languages including node.js, Python and C#. Node.js works well alongside MongoDB and Angular which is shown in the MEAN stack meaning it was the best choice of language for this project.

2.1.4 Ionic

Ionic was used in creating the Surf Swell Check Android app. It is a framework that uses Angular along with its own library of UI components which differ from Android to iOS, making it easy to release the app on either platform. The only issue with releasing on iOS is that it is not possible from a Windows machine.

The Android app uses Ionic because it is admired by a lot of developers for its free and easy to use nature. "IONIC has streamlined the development of multi-platform applications, bringing a classy feel to the space of hybrid apps" (INKONIQ, 2017). Testing is also easy as the app can be tested in a desktop browser, making it very easy to develop and immediately see results. It can also be tested on an emulator and easily installed on a real Android device from an APK file.

2.2 Chatbot

"A chatbot is a service, powered by rules and sometimes artificial intelligence, that you interact with via a chat interface. The service could be any number of things, ranging from functional to fun, and it could live in any major chat product (Facebook Messenger, Slack, Telegram, Text Messages, etc.)" (Schlicht, 2016). In the case of Swell Surf Check, the chat bot on the Web Application uses artificial intelligence in the form of natural language processing to work out what the user is asking. There is then code that fulfils the task requested by the user. The bot learns with each question asked and becomes more accurate over time. At the moment, the chat bot is integrated into the Web Application as well as Slack.

Google's Dialogflow was selected as the bot framework to be used within Swell Surf Check. "Dialogflow is user-friendly, support 20+ languages, and probably the best framework to develop NLP based applications" (Rehan, 2019). Dialogflow makes use of natural language processing (NLP) to learn what the user is trying to ask. "Natural language processing employs computational techniques for the purpose of learning, understanding, and producing human language content" (Hirschberg & Manning, 2015). It would be impossible to guess the many ways a user could ask a question so natural language processing is important in finding the user's intent in order to fulfil their needs.

2.3 Android HCI

User interface is one of the most important things to consider when designing any app. The Swell Surf Check Android app used the official Android developer guidelines, some of which are shown in Table 1, to ensure the app was designed with the end user in mind.

Table 2 - Android guidelines on visual design and user interaction (Android, 2020)

Area	Description
UI and Graphics	The app supports both landscape and portrait orientations (if possible). Orientations expose largely the same features and actions and preserve functional parity. Minor changes in content or views are acceptable.
User/app state	The app should not leave any services running when the app is in the background, unless related to a core capability of the app. Learn how to choose the right solution for your work. For example, the app should not leave services running to maintain a network connection for notifications, to maintain a Bluetooth connection, or to keep the GPS powered-on.

2.4 Web HCI

The Web Application was built using the Bootstrap framework to follow the recognised web design guidelines. According to their website, Bootstrap is “the most popular HTML, CSS, and JS library in the world” (Bootstrap, 2020).

“Using Bootstrap, you can save a lot of time. You don’t have to spend time writing code, you can just use the Bootstrap predefined design templates and classes and put it exactly where they fit” (Caliman, 2015). This was useful in the development of the Web Application as time was not wasted trying to stick to the same styles throughout the website, it was all done by Bootstrap. Its responsive nature was also a big reason for its use, allowing for automatic adaption depending on screen size so users can easily access the Swell Surf Check Web Application on a desktop, tablet or mobile.

2.5 Existing Product Analysis

2.5.1 Magic Seaweed

Magic Seaweed is an existing surf forecaster who use their own data to predict the surf conditions at surf spots around the world. “Magicseaweed is the world’s largest online surf forecasting platform with 2m+ users a month, providing weather and wave forecasts over 5000 beaches around the world” (Hickman, 2019). It is available on web, Android and iOS as shown in Figure 1.

Figure 1 - Magic Seaweed mobile apps (Magic Seaweed, 2020)

While Magic Seaweed has information on thousands of surf spots, Swell Surf Check focuses on fewer spots, giving more information about each, rather than a brief overview of many. Swell Surf Check allows users to see points of interest in the surrounding area, helping newcomers to get to know the area better rather than just helping surfers find the surf conditions.

Swell Surf Check also provides the surf conditions in a more innovative way, in the form of a chatbot. This feature is currently not supported by Magic Seaweed and aims to provide the user with an easily accessible and fast way to check the conditions while on the go.

3. Method of Approach

Swell Surf Check was developed in an agile environment. The agile manifesto, published in February 2001, outlines the principles behind agile development. “Agile is an iterative approach to project management and software development that helps teams deliver value to their customers faster and with fewer headaches. Instead of betting everything on a "big bang" launch, an agile team delivers work in small, but consumable, increments. Requirements, plans, and results are evaluated continuously so teams have a natural mechanism for responding to change quickly” (Atlassian, 2020).

Scrum is an approach to the agile framework. “Scrum encourages teams to learn through experiences, self-organise while working on a problem, and reflect on their wins and losses to continuously improve” (Atlassian, 2020). There are several events associated with scrum, the first being sprints which are usually 2-4 weeks long. The sprint encapsulates the other events (shown in Figure 2) into a time frame.

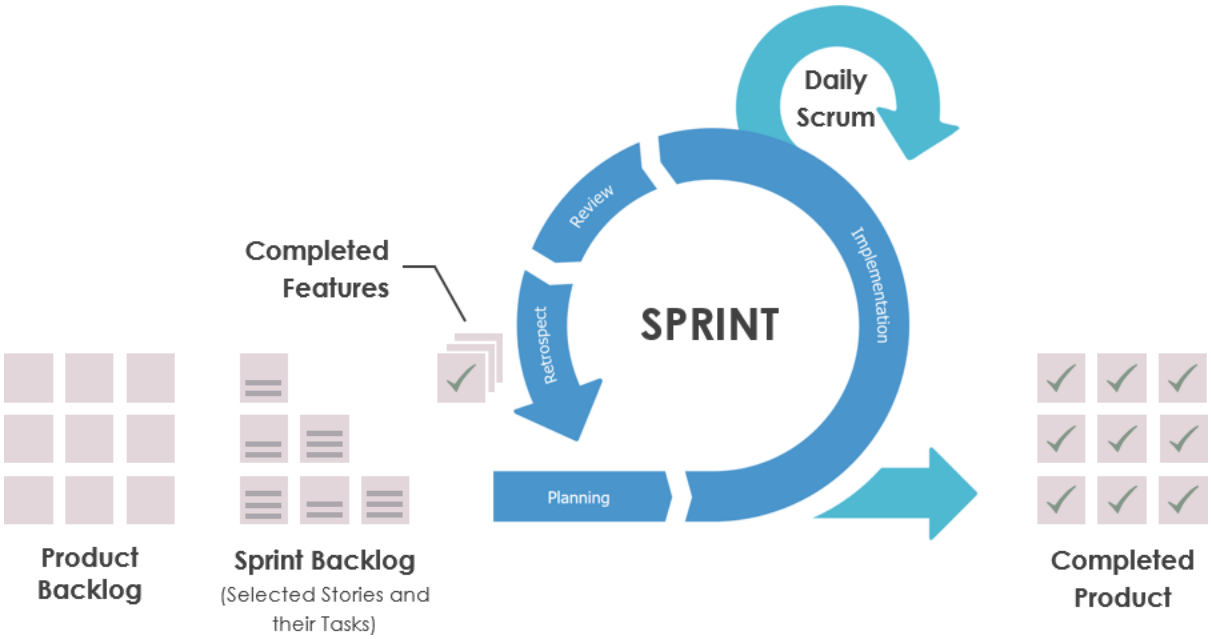


Figure 2 – Sprint (Visual Paradigm, n.d.)

The sprints in the development of Swell Surf Check were 2 weeks long. This was chosen as the length so a substantial amount of work could be done given the time frame of the project.

The project started out with a product backlog where small tasks were created from user stories and put into a list. Before each sprint started, several items from the backlog would be brought into the sprint and this would be the work to be completed within the next two weeks. Any work that had not been completed was brought into

the next sprint. Because each sprint was focused on a piece of functionality, there was the benefit of having a releasable product at the end of each sprint.

Before development started on the project, two initial sprints were completed which involved creating the planning documentation so that once started, the project had a definitive direction to go in.

Trello was used to visualise each sprint and separate into 3 columns, as shown in Figure 3. “Trello is a collaboration tool that organizes your projects into boards. In one glance, Trello tells you what's being worked on, who's working on what, and where something is in a process” (Trello, 2020). This visualisation of the work that needed to be done, what was in progress and what was already done, made the project a lot less daunting and clear progress can be seen with the screenshots of each week’s board. These can be viewed in appendix 15.2.1.

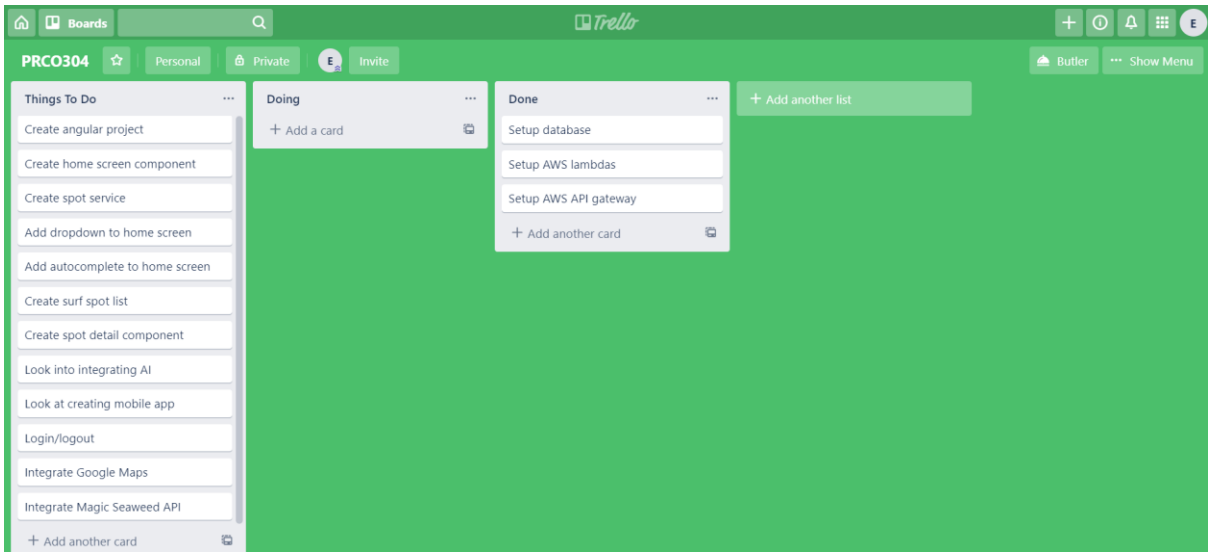


Figure 3 - An initial idea of the tasks to be done

4. Legal, Social, Ethical and Professional Issues

When creating a new product like Swell Surf Check there are several legal, social, ethical and professional issues to think about. One of these being user data protection. The Data Protection Act (gov.uk, 2018) outlines the numerous measures that must be put in place to ensure the safety of user’s data. In the case of Swell Surf Check, user’s information is only stored when they choose to create an account where the user’s name, email, username and password is stored. Creating an account is optional and the user can still use the app without having to sign up.

User data is stored in the database when the user creates an account. The chosen password is hashed and salted using a hashing algorithm provided by a node.js library – Bcrypt. This way the hashed password stored in the database cannot be stolen by third parties. “Cracking a strong, encrypted password can take decades and ample supercomputing power to guess” (Spy Cloud, 2019). Meaning user’s data within Swell Surf Check should stay secure.

The Google Places API has a policy that must be followed when in use by anyone. In the policy they state, “If your application displays data from the Places API on a page

or view that does not also display a Google Map, you must show a Powered by Google logo with that data" (Google, 2020). The logo that should be displayed is shown in Figure 4. Swell Surf Check aims to abide by the terms and conditions set out by the third-party services it uses.

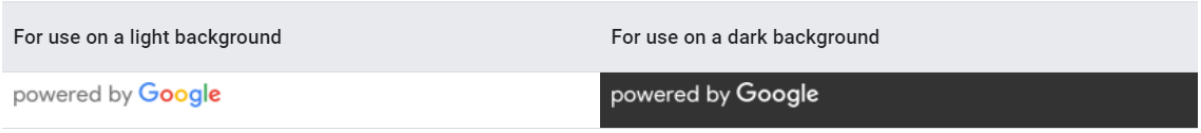


Figure 4 - Powered by Google logo (Google, 2020)

Throughout the development of Swell Surf Check there were multiple usability testing sessions that took place to build on the quality of the application. While it is important to get as much information out of these sessions as possible, "we should refrain from exerting too much stress on the subjects. Excessive intensity, stress, or realism simply isn't ethical" (Leffingwell, 2017). To avoid users becoming stressed or frustrated during the testing, the users were treated with respect throughout and it was made clear that nothing they said could be considered wrong. This also helped to get a more honest picture of the overall usability of Swell Surf Check. The testing was planned well in advance and users were made fully aware of what the sessions would consist of before agreeing to take part so there would be no shock during the sessions.

5. Project Management

"A goal of any software project management is to develop/maintain a software product by applying good project management principles as well as software engineering principles so that the software project is delivered at minimum cost, within minimum time, and with good product quality" (Ahmed, 2011). Therefore, if a project is managed well, it should be completed within a given time frame and ensure the final product is to a high quality.

Swell Surf Check was managed using several management ideas and software had a big part to play, some of which is listed below.

5.1 Supervisor Meetings

At the beginning of development, supervisor meetings took place on a weekly basis, starting with outlining the initial idea and how development was planned. Having meetings in a group was beneficial because it was useful to know what stage others were at with their projects and to discuss different technologies that they might be using to get a broader knowledge of the kinds of technologies that could have been used for Swell Surf Check. Once development had started, the meetings had a set routine of going around the group and explaining what had been achieved that week, what was planned for the next week of work and how that would be achieved.

The meetings were invaluable during the early stages of Swell Surf Check with the guidance of a professional that set the project on the right track.

5.2 Version Control and DevOps

Git was used for the version control throughout the project. “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later” (Git, 2020).

A repository was set up for both the Web Application and the Android Application on GitHub. Command line instructions were used to push the work on a branch to the relevant repository where a pull request would be produced and the changes in the code could be looked over before merging the branch into the master branch.

There were many benefits of using version control during the development of Swell Surf Check. For example, if something were wrong with the code, it could simply be reverted to the master branch and the branch with broken code could be removed. Furthermore, it safeguards against data loss and keep a record of code and commits.

The code editor used for the Angular front-end was Visual Studio code. This was used because it has Typescript language support and an integrated terminal so using the Angular CLI was made easy. The back-end AWS Lambdas were also written in Visual Studio code and then uploaded to the AWS console via a .zip file. This was necessary because there were dependencies such as MongoDB that needed to be installed and this cannot be done in the console.

The pipeline worked well because a small amount of code would be submitted to the GitHub repository at a time so if any bugs were found it was easy to revert back to the previous version. Because there was only one person coding the website, it was not necessary to have multiple branches so everything was merged on the master branch and the pipeline that was setup in Heroku automatically builds the code from the GitHub repository and deploys the website to a public URL. Ideally, the tests would automatically run before Heroku is allowed to deploy the website so there is a lesser chance of deploying broken code, however this is a paid feature of Heroku.

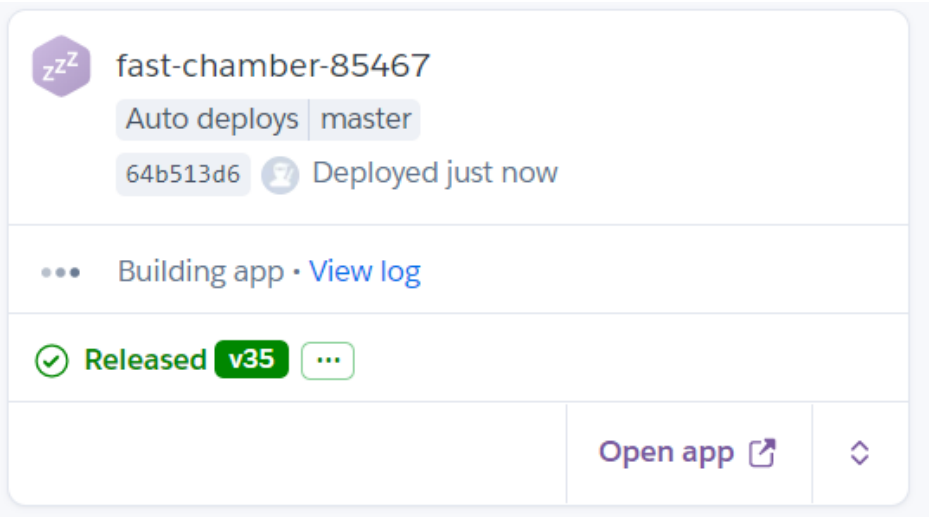


Figure 5 - Heroku auto build and deploy from GitHub, including version number.

5.2 User Management

User management was an important aspect of the user testing so that those taking part had clear instructions on what to do so that they felt confident and positive about their experience. Users being confident in expressing their thoughts was important in obtaining valuable feedback to better the final product.

5.3 Time Management

To ensure the project was delivered within the given timeframe, it was essential to have a well thought out plan which included how the time would be managed. Several time management techniques were used during the development of Swell Surf Check.

During one of the first supervisor meetings, it was highly recommended to have a well thought out time plan of what should be done by when in order to stay on track for a timely delivery of the project. Therefore, a high-level time plan was created that roughly outlined the dates that certain features should be ready by. While there was slight deviation from the plan during development, it was for the most part stuck to, and this helped to keep stress levels down and productivity high. This was also in keeping with the agile methodology used in the project where if some functionality were not completed on time it would simply be pulled into the next sprint.

6. Requirements

The requirements in this section outline the functionalities required by the project and what priority they stand at. Priority is a number between one and four, one meaning the requirement must be in the system and four meaning it would be an addition if there were enough time and resources, with two and three being the middle ground.

6.1 Functional Requirements

Table 3 - Functional requirements

Requirement	Priority
The user should be able to login with a button on the nav bar in the Web Application and a button in the settings of the Android Application.	2
The user should be able to register an account with a chosen username and password.	2
The user should be able to see their account details on a profile page.	2
The user should be presented with a forgotten password link in the case they forget their login credentials.	4
The user should be presented with a toast if the credentials they enter are wrong.	2
The user should be able to change their password.	4

The user should be presented with an error message if they try to register with invalid credentials.	2
The user should be presented with an error message if the username they try to register with is already taken.	3
The user should be presented with a home screen that they can search and select surf spots from.	1
When the user clicks the Swell Surf Check icon in the Web Application, they should be redirected back to the home screen.	2
In the Web Application, the user should be able to search for a spot with the aid of autocomplete.	2
The user should be presented with a spot detail page after choosing a spot.	1
The spot detail page should include the current surf conditions.	1
The spot detail page should include a forecast of future surf conditions.	1
The spot detail page should include nearby points of interest.	2
The nearby places of interest should be placed on a map.	2
When clicking on a point of interest, the user should be presented with a modal with more information.	2
The information modal should include a gallery and reviews.	3
The spot detail page should include what conditions for each spot is best.	3
Magic Seaweed's API should be integrated to show charts and other advanced information.	3
The user should be able to ask a chat bot what the conditions are doing at a spot.	2
The user should be able to ask a chat bot which spot has the best conditions around them.	4
The user should be easily able to navigate between the chat bot, surf forecasts and profile pages easily.	1
The user should be able to add a spot to their favourites for easy access from their profile page.	3

The user should be able to remove spots from their favourites.	4
The UI should be intuitive.	1
The user should not be able to access the profile page unless they are logged in.	1

6.2 User Requirements

Table 4 - User requirements

Requirement	Priority
The user will be able to register for an account.	2
The user will be able to login to the Swell Surf Check on both the Web Application and the Mobile Application.	2
The user will be able to view their profile on a profile page.	2
The user will be able to search for a spot from the home screen.	1
The user will be able to browse spots based on a region.	1
The user will be able to search for a spot with a user entered search string.	1
The user will be able to select a result based on their search string from an autocomplete dropdown.	2
The user will be able to see spots based on their search in the form of a list.	1
The user will be able to view the current surf conditions of a spot.	1
The user will be able to see the future surf conditions of a spot.	1
The user will be able to see nearby points of interest in the form of a list.	2
The user will be able to see nearby points of interest on a map.	3
The user will be able to add a spot to their favourites when logged in.	2
The user will be able to delete a spot from their favourites when logged in.	4
The user will be able to see their favourite spots from the profile page when logged in.	2
The user will be able to ask a chatbot what the surf conditions are doing at a spot.	1

The user will be able to view the best conditions for a spot.	2
The user will be able to see advanced surf forecast information.	3
The user will be able to see advanced charts that predict different aspects of the surf.	3

6.3 Non-Functional Requirements

Table 5 - Non-functional requirements

Requirement	Priority
There should be a backup of the source code.	1
The usability testing should say the applications are easy to use.	1
The applications should be accessible to those with disabilities.	2
The system should be testable via automated testing.	1
Response times for user interaction should not exceed 5 seconds.	1
The code should be easily modifiable and adhere to known coding practices.	1
The system should be secure from hackers, e.g. by hashing and salting passwords.	1
There should be documentation for the application.	2

7. Architecture

There were many options for the architecture of Swell Surf Check. The final choices are documented in the section below.

7.1 Architecture Overview

An outline of the high-level architecture for Swell Surf Check is shown in Figure 5 below.

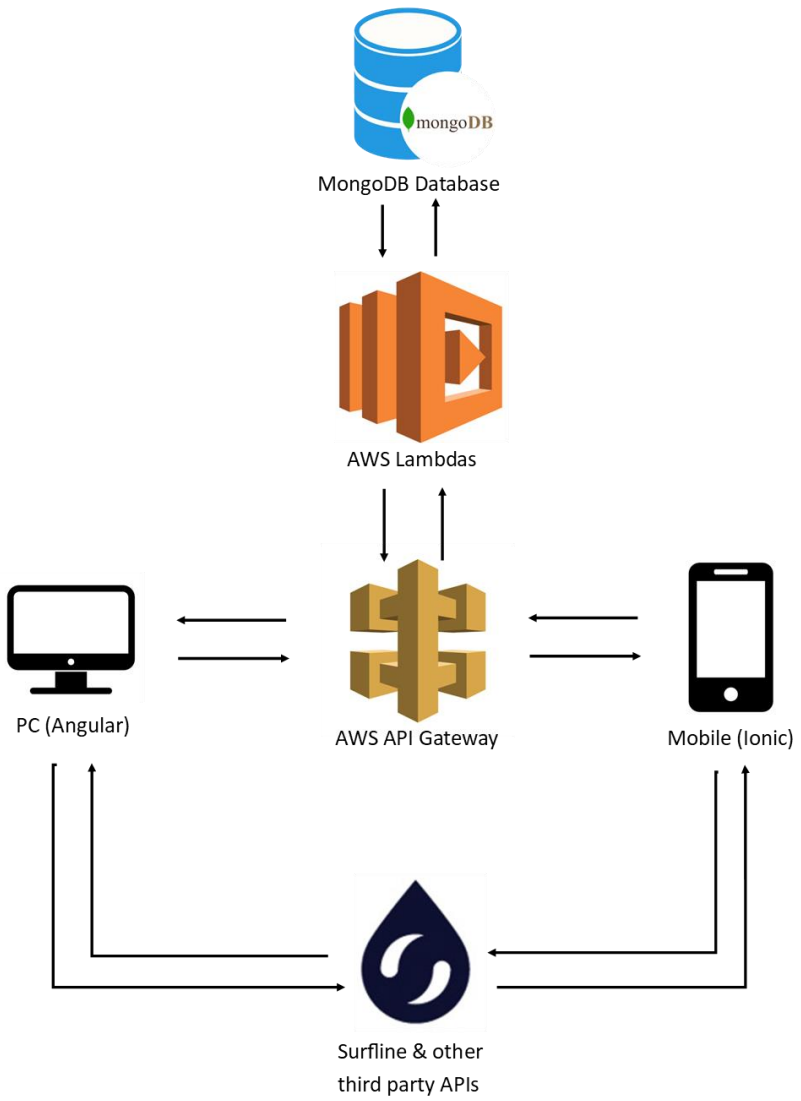


Figure 6 - High level architecture diagram

The figure shows that the client devices communicate through the AWS API Gateway to the AWS Lambdas which communicate with the database. The API Gateway is responsible for creating the end points for each Lambda that can be called from each of the client devices to get the data they need.

The client devices also directly communicate with the third-party APIs to get the data for the surf reports which is processed and displayed within the client.

7.2 Database Schema

The database schema is as shown in Figure 6. Because of the choice of MongoDB for the database, the schema does not resemble a classic entity relationship diagram. MongoDB is a document-based database meaning it does not have features that a classical database has, for example foreign keys.

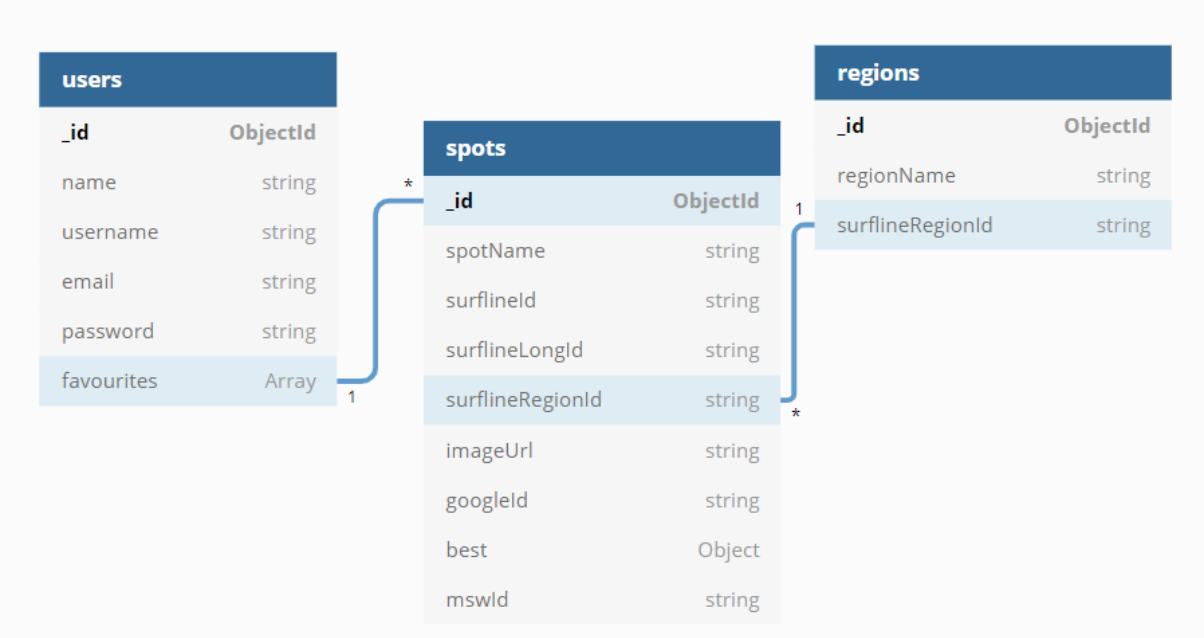


Figure 7 - Database schema

7.3 UML Diagrams

Figure 7 shows a potential flow through the Web Application to add a surf spot to as user's favourite spot while Figure 8 shows adding a spot to favourites as a use case.

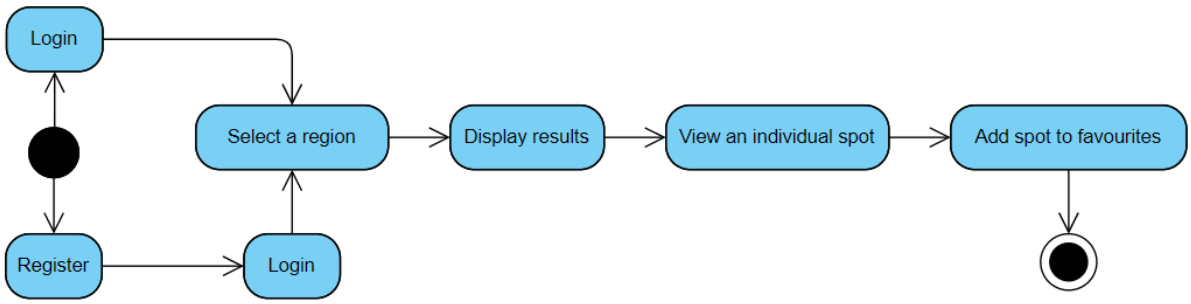


Figure 8 - Activity diagram: add a spot to favourites

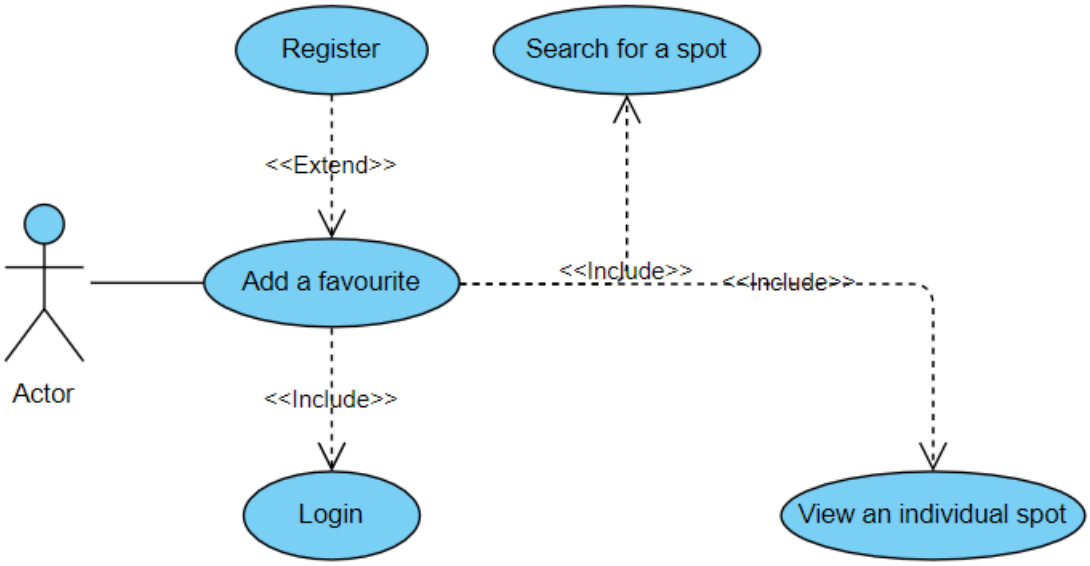


Figure 9 - Use case: add a spot to favourites

Further UML diagrams that show the structure of Swell Surf Check can be found in appendix 15.3.

8. Design

This section will overview the design of Swell Surf Check including the initial protype design and that of the Web and Android Applications.

8.1 Initial User Interface Design

The initial designs of the user interfaces for both the Web and Android were drawn up on paper first to work out where everything fitted best. These are known as wireframe designs. “Wireframes are simple black and white layouts that outline the specific size and placement of page elements, site features, conversion areas and navigation for your website” (Haas, 2011). It was a beneficial way to make a start on how the applications would look without having to commit to anything or waste time coding unneeded features.

8.2 Prototype Development

Before a final design was settled upon, Web and Android prototypes were produced to ensure the technologies chosen were able to work together. Prototyping is known to create higher quality products. “Prototyping is an integral part of Design Thinking and User Experience design in general because it allows us to test our ideas quickly and improve on them in an equally timely fashion” (Dam & Teo, 2020). This way a proof of concept could be delivered without having to spend a large amount of time coding unnecessary parts. On the other hand, no time was wasted coding an idea that would not work in practice as this would have been ironed out in the prototype.

Both the Web Application and the Android Application prototypes confirmed that the API data could be retrieved through the client and displayed to the user. However, it was found out that, while a live stream of the beach worked locally, when deployed the request was blocked by CORS. As this was an issue on the server side of the third-party API, there was nothing that could be done so instead just a photo of the beach is displayed. With this issue becoming apparent within the prototype, not much time was spent coding based on having a live stream, therefore no time was wasted.

8.3 User Testing

User testing is important not only to find bugs within a system but also to ensure the user experience is the best it could be. “You can have the most beautiful website in the world, but people will leave immediately if they are unable to figure out how to navigate your site quickly” (Churm, n.d.). Having real people test the system can help massively in finding the best layouts and routes through it.

Because of the iterative nature of agile, Swell Surf Check was developed in a way that improved the user interface over time. To aid this, user testing was carried out throughout the project. This is called explorative testing which is “used early in product development to assess the effectiveness and usability of a preliminary design or prototype, as well as users’ thought processes and conceptual understanding” (Churm, n.d.). This early testing happened once prototypes had been established and the prototypes were then developed from the user feedback. The second method of testing employed in the project was assessment testing which took place towards the end of development to get an overall view of user’s opinions on the finished product.

The data produced from the testing was analysed to find common areas where the users agreed the interface could be improved. Users were also able to add any ideas they had about requirements that could be added as well as confirming existing requirements were being met.

Users that participated in the testing sessions were from a wide variety of backgrounds, including surfers and non-surfers in order to get a well-rounded picture of information.

8.3.1 Web Application

During the first stage of testing, a prototype of the system was presented to users. This early model had only the barebones of the application and from the feedback of the users it was clear there were flaws with the system, with only 50% of users agreeing the system was intuitive and easy to use (Figure 9).

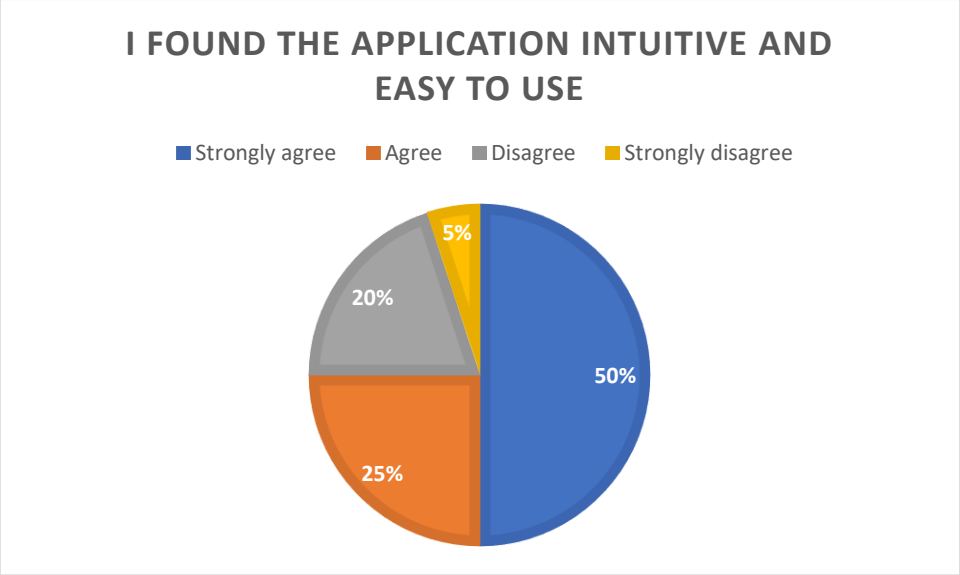


Figure 10 - Questionnaire results

Using the feedback on the prototype from the users, an easier to use and more intuitive system was created (Figure 10).

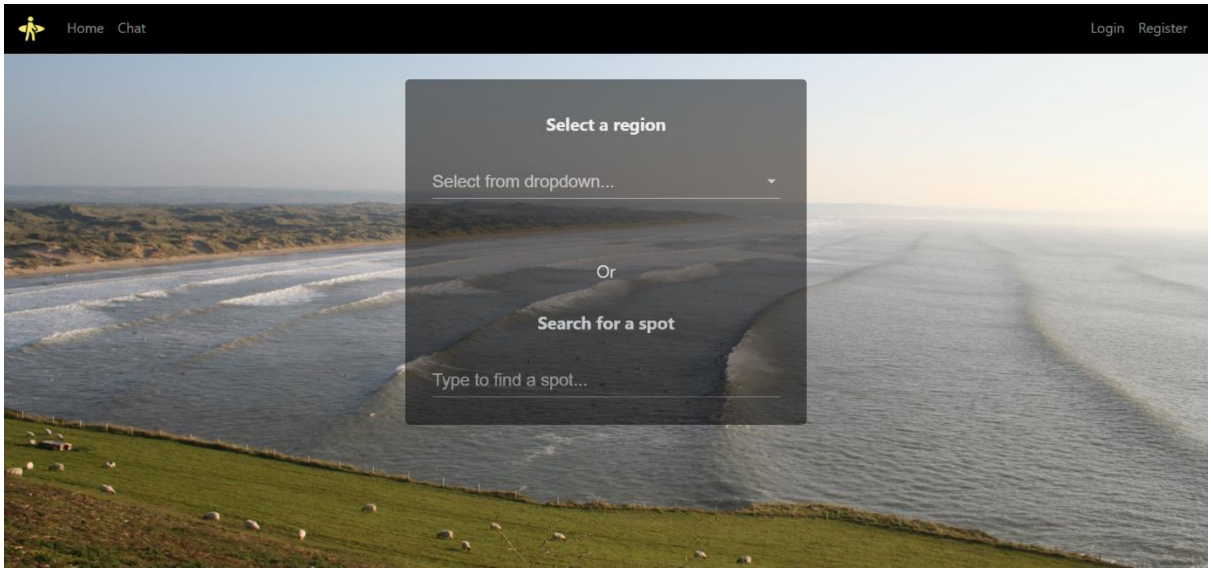


Figure 11 - Improved home screen

In the initial prototype, there was a simple search box on an empty background. User's reported this was hard to see and was not appealing. "Internet users are accustomed to being able to figure out how to use a website quickly. Most of them will not take the time to figure out a site that is not usable" (Churm, n.d.). Therefore, the improved home screen was created with an aesthetically appealing background and an easy to use search box that has multiple search options for users of different surfing backgrounds.

The second round of usability testing asked the users to perform tasks within the system, for example adding a spot to their favourites. From the feedback given the spot detail page was perfected. The resulting interface is shown in Figure 11.

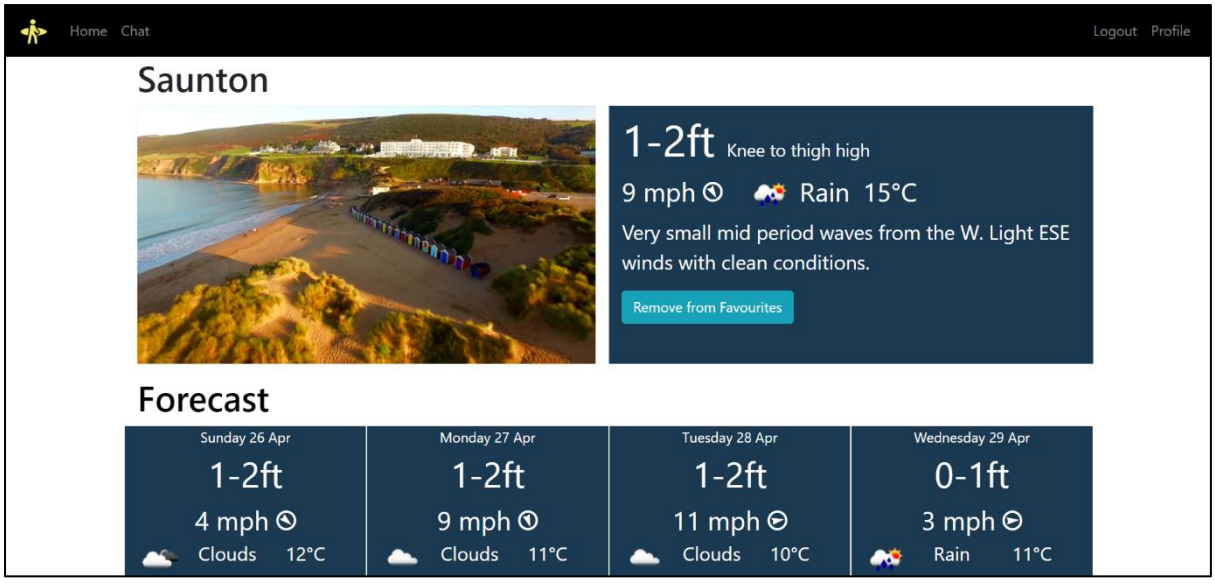


Figure 12 - Improved spot detail page

After using the improved system, 100% of the users agreed that the system was more aesthetically pleasing and easy to use.

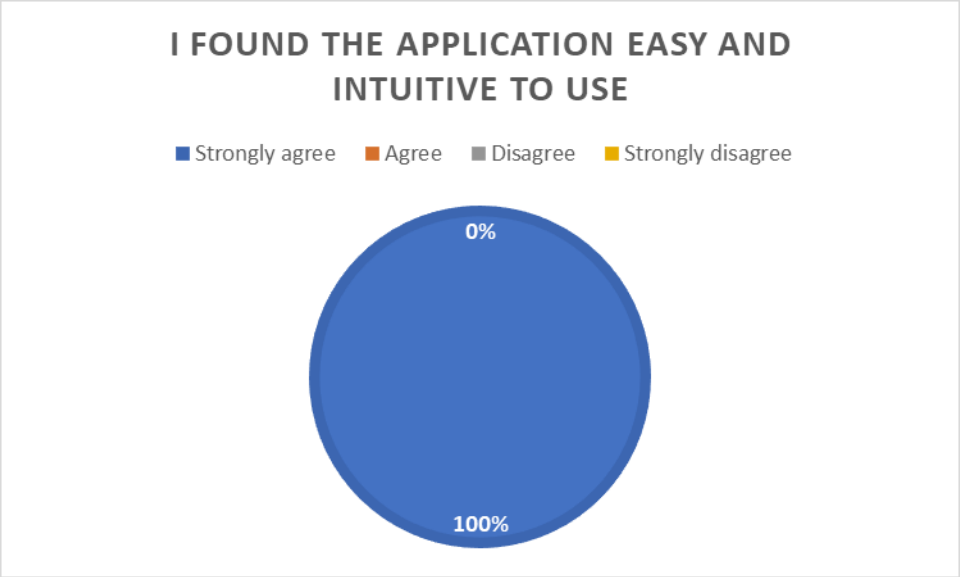


Figure 13 - Improved questionnaire results

8.3.2 Android Application

The user testing of the Android Application was like that of the Web Application where users were given set tasks to do. This gave several improvements to be made to the interface. While the prototype interface consisted of just a list of spots, users thought there should be more colours to be more attractive. Only 70% of participants agreed the system was intuitive and easy to use at this point (Figure 13).

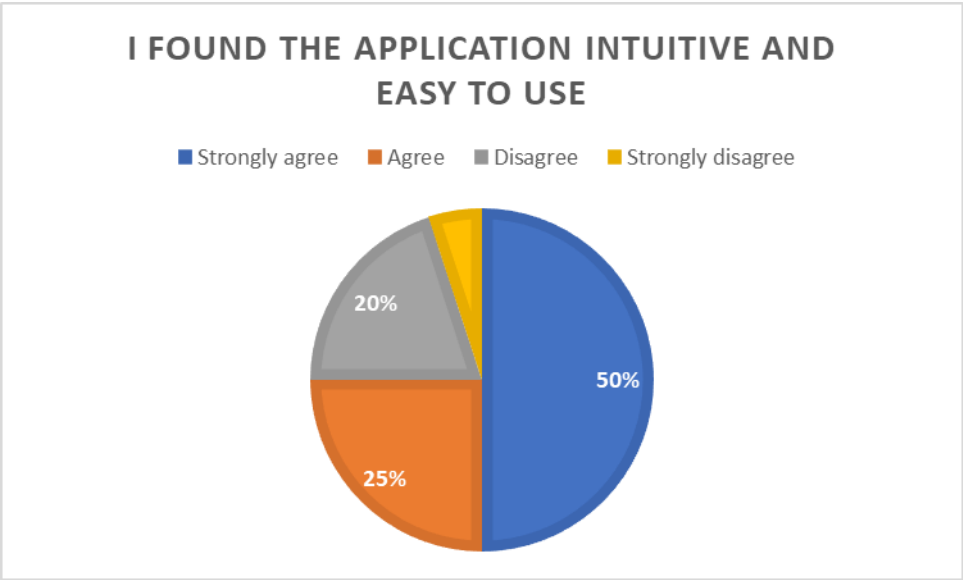


Figure 14 - Questionnaire results

This resulted in more colourful pictures to be added so users would be more attracted to the application (Figure 14).

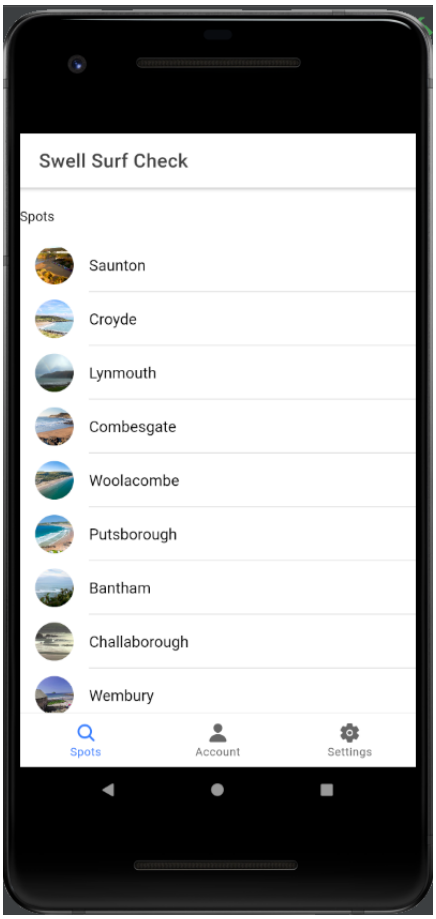


Figure 15 - Improved mobile home screen

Once the new, more functional interface was tested on the users, 100% of them agreed that it was now intuitive and easy to use (Figure 14).

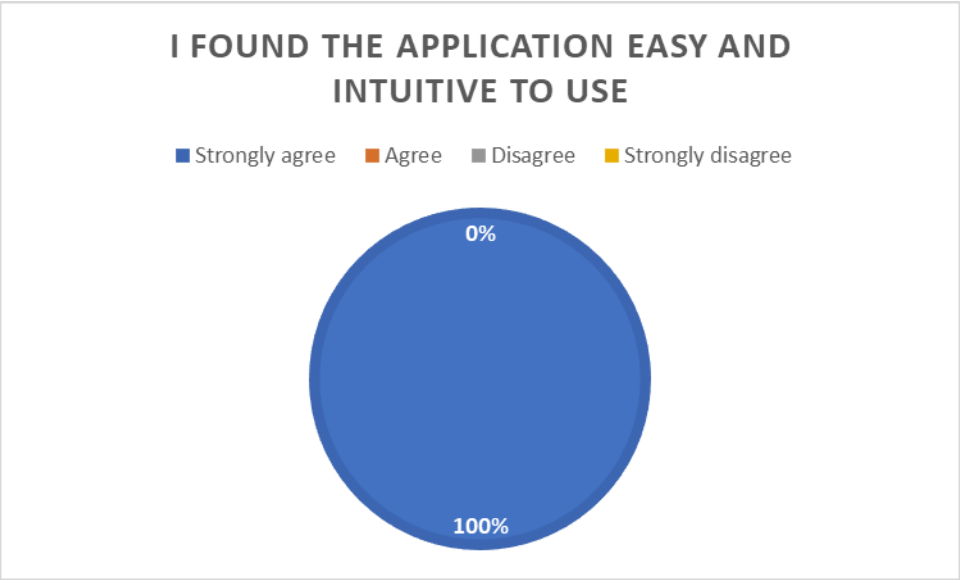


Figure 16 - Improved questionnaire results

9. Project Stages

This section describes each stage of the project and goes into detail about each sprint that took place. Each stage will provide a summary, an aim of deliverables as well as the key challenges and accomplishments.

9.1 Stage 1

9.1.1 Summary

Stage 1 consisted of deciding on the exact nature of the project and what key features needed to be included to find the right scope and to make sure the main goals were not too much or too little. While there were no formal documentation or deliverables for this stage, it was a vital steppingstone in making progress at the beginning of the project. Here it was decided the main aspects of the project would be to deliver a Web Application, an Android Application, and an AI chatbot.

9.2 Stage 2

9.2.1 Summary

The main goal of stage 2 was to conduct research into the best technologies to use for the different aspects of the application. During this stage, many prospective technologies were researched, outlining the pros and cons of each, and ensuring the technologies worked well together before making a final decision.

Stage 2 was also where the requirements were decided and whether the technologies chosen would help to fulfil these requirements.

After the requirements were decided, a rough Web and Android prototype were developed in order to see the chosen technologies working together and for use in the initial usability session.

9.2.2 Deliverables

- Document outlining requirements
- Document outlining research for chosen technologies

9.2.3 Key Challenges and Accomplishments

The initial plan to use the Magic Seaweed API to provide all surf data did not pan out when only two surf spots were available for use. The system relies on having data for multiple surf spots in the South West and two was not enough. For this reason, it was decided the Surfline API would be used for most of the data. However, this API is not documented, and access could be restricted at any time so put the reliability of the system at risk. Weighing up the pros and cons, it was decided that this was the only way forward.

9.3 Stage 3

9.2.1 Summary

Once the technologies for Swell Surf Check were chosen, it was possible to create a high-level architecture diagram to begin to think about how the technologies would fit together and how the third-party API data would be used.

During this stage, user stories and UML diagrams were also drawn up to visualise the different functionalities users will need and how they can be achieved.

The database schema was also visualised in a schema diagram.

9.2.2 Deliverables

- High level architecture diagram
- Database schema
- UML diagrams

9.4 Stage 4

9.4.1 Summary

This stage is where the first sprint took place. The aim of the sprint was to develop a basic interface for both the Android Application and the Web Application, building on what had already been done in the initial prototypes. The basic interface included a home screen that would welcome users as well as an easy to access nav bar where users could log in and out, register a new account, and view their profile.

To enable users to log in and out, forms were created where users could enter their credentials. To enable a user to register, validation checks are performed on the registration form to ensure the user's email address is valid and all fields are filled out before submission to avoid breaking the system. Once a user submits a registration form, a POST request is sent to the AWS API Gateway endpoint where a Lambda is used to connect to the MongoDB database that was set up for the prototyping. Once connected, a new user document including the user credentials and hashed and salted password are stored in a remote instance of the database. When the user attempts to login with their registered credentials, another POST request is sent to an authenticate endpoint in the AWS API Gateway and if the credentials match those in the database, the user is logged in. Once logged in the user's credentials are stored in the browser's local storage for the Web Application

and Ionic local storage for the Android Application as well as a JWT token that is created on login. This allows the user to be logged in for a week before having to re-enter credentials if not manually logging out.

A profile page was also created so that users would be able to add surf spots to their favourites and have a place to view them and quickly navigate to the spot they want.

This sprint is where the initial user testing was conducted to get an early idea of the thoughts of users. From this it was learned that the interface was not intuitive, and it was difficult to find where to search for spots. This insight made it possible to shape the applications around the user's needs and well as gain more possible requirements for future versions of the applications.

9.4.2 Deliverables

- RESTFUL API for authentication
- Login and registration functionality
- User interface usability feedback

9.4.3 Key Challenges and Accomplishments

A RESTFUL API was created allowing users to register an account as well as log in and out of the system on both the Web Application and the Android Application.

Protecting user's personal information was the main challenge in this sprint as unauthorised individuals could try to get a hold of the information. To overcome this, instead of passwords being stored in plain text within the database, they are subject to a hashing algorithm using BCrypt – a Node.js hashing library. An auth guard was also added to the profile route in the Web Application so a user can only access the profile page if they are logged in to prevent unauthorised users making requests to the database.

9.5 Stage 5

9.5.1 Summary

Stage 5 is where the second development sprint took place. This sprint focused on the spot search functionality within the Web and Android applications.

User's in the usability session made a point of the search being difficult to use and only having one kind of search option, so it was decided users would have a choice of picking a region in order to browse a relevant list of spots based on their area, or an autocomplete search box where users would begin to type a spot name and ideas based on what they typed would appear for them to go directly to a spot's forecast page. To enable this, firstly a regions and spots needed to be added to their respective collections the database, with one region having many spots. To affiliate a spot with a region, a region ID was added to the spots of the region they belonged to. It was found out at this stage that to make a request to the Surfline API, a Surfline spot or region ID was needed so the Surfline region ID became the link between spots and regions within the database.

Next, a spot service was set up in the client side Angular and Ionic projects that sends HTTP requests to the database and, later, the third-party APIs to get the information needed.

Once the database was set up with spots and regions, a dropdown window was created on the home page that filled from the regions in the database. When a user clicks on their chosen region, they are taken to a spot list page where all the spots within that region are displayed.

After this search functionality was achieved, the autocomplete search box was added. From research, it was decided that Material Design had the best autocomplete search box feature built into their public UI components library, so this was used to aid the set up. Using a GET request from the spot service, the spots were placed in an array on the client side and a filter was set up so when a user typed a letter into the search box, only spots containing that letter would be displayed. When a spot is clicked, the user is taken straight to the spot detail page.

It was decided at this point that the Android Application would be more of a lightweight version of the Web Application. To avoid a too complex user interface on mobile, instead of a search function, only a list of spots is displayed that the user can choose from to go straight to the spot detail page.

9.2.2 Deliverables

- User interface improvements from previous usability session
- Spot service in Web Application
- Spot service in Mobile Application
- Web search functionality

9.2.3 Key Challenges and Accomplishments

All search functionality was completed, with a solid foundation of database data ready to make API calls to Surfline. The layout of the homepage was improved based on user feedback to make it more intuitive and easier to use. A spot list page was also added to display spots associated with a certain region so users who do not know exactly which spot they are looking for are able to browse based on their location.

It was discovered that when in the spot list page or the spot detail page in the Web Application and the page was refreshed, the system would forget where the user was and taken back to the home page. This was not very user friendly as a user could lose the progress of their search in the case of a page refresh. To counter this, a region ID path parameter was added to the route of the spot list page and a spot ID path parameter to that of the spot detail page. This way when the page is refreshed the user is not taken back to the home page, instead the same search results or spot they were looking at are still displayed.

9.6 Stage 6

9.6.1 Summary

Stage 6 consisted of the third sprint that was centered around the development of the spot detail page of both the Web and Android Applications which would include

the current surf conditions as well as a forecast of future conditions. This sprint also included adding the nearby points of interest feature with a map depicting where each place was.

With the work done in the previous sprint to get the spot ID to appear in the URL of the page, it was relatively easy to get the URL, feed it into a Surfline API call in the spot service and dynamically fill the page with data depending on the spot chosen. To display the data in an aesthetically pleasing and responsive way, Bootstrap's grid system was used. This splits the screen into 12 equally spaced columns and, depending on the size of the user's screen, the number of columns that a certain div takes up can be set. This allowed for easy display of the data coming from Surfline.

It was decided at this stage, it would be beneficial to the user that not only the surf conditions be displayed, but also the weather. This was done in a similar way to the Surfline API call, but instead went to the Open Weather API, through a newly created weather service on the client side.

Once the surf and weather were successfully displayed, the Google Places API was used to find nearby points of interest from the chosen spot. The API call consists of a longitude and latitude which is provided by the Surfline data, a search radius which was set to 3000 meters, and a keyword for the type of points of interest found, in this case "surf" was used. Each nearby place was pushed to an array within the Angular front-end from which a marker was added to a Google Map on the page. An icon was added to each nearby place in the list displaying them so the user could click the icon and the relevant marker on the map would be highlighted so users could find out exactly where the place was from the spot they were at.

Because the Google Places API provides so much data, it was decided that if it were all displayed at once, the page would become too crowded and overwhelming for the user. Hence, a modal was used so that when a place name was clicked from the list, the modal would appear on top of the other content on the page and would include pictures, address and reviews from other Google users.

The final component to add to the spot detail page was a section for surfers that wanted more advanced information on the current surf conditions. This information was retrieved from Magic Seaweed as it was not available from the Surfline API. This API is not public so permission from Magic Seaweed had to be acquired. However, they gave access to only two surf spots so this information can only be displayed on the detail page of these spots only. Within this component there are charts of different aspects of the surf, which are displayed in a gallery to match the style of the rest of the Web Application.

This sprint is where the second iteration of user testing took place. Users commented that the system was much easier and more intuitive to use after initial testing session and subsequent changes to the UI.

9.6.2 Deliverables

- Current conditions component in Web Application
- Current conditions component in Android Application
- Forecast component in Web Application

- Forecast component in Android Application
- Nearby spots component in Web Application
- Magic Seaweed component in Web Application
- Final user feedback

9.6.3 Key Challenges and Accomplishments

The spot detail page was completed for both the Web and Android Applications including current conditions, forecast, nearby and Magic Seaweed components. Google Maps was integrated into the Web Application with numerous third-party APIs communicating with the client to display the relevant information.

The main challenge in this sprint was the UI design, getting each element in the correct place and ensuring the Web Application was responsive so would scale down to smaller screens, for example on a mobile web browser. To achieve this, the Bootstrap grid system was used.

9.7 Stage 7

9.7.1 Summary

Stage 7 was where the fourth and final sprint took place. This sprint consisted of the development of the AI chatbot. After initial research it was decided that Google's bot framework – Dialogflow – would be used. The first step was to set up the intent. Only one intent was needed for the project – Check Surf. To enable the bot to learn, training phrases were added, for example, "what is the surf doing at Fistral?" and "how is the surf at Saunton?". Around ten training phrases were added, and this seemed to allow the bot to learn so that when anything similar was asked by a user, it would now recognise it as the Check Surf Intent. The bot also learns from phrases a user inputs to train itself on user's intents.

Once the chatbot understood what the user was asking for, webhook fulfilment was set up to fulfil the user's request. This was done in the inline code editor within Dialogflow. It creates a web service that receives a POST request from Dialogflow containing the user's intent. The webhook is in Node.js and first uses an AWS Lambda that connects to the MongoDB to get the Surfline ID of the spot the user has requested given the spot name. Next, it uses the Surfline ID to make a GET request to the Surfline API to get the current surf conditions. All of this functionality was put into a function which is called when an intent map maps the Check Surf intent to the function so that when the intent is triggered, the function is called and the surf conditions are displayed to the user via the chat agent. Error handling was then added to ensure a user would always get a response, whatever they input as well as adding the Dialogflow prebuilt small talk agent so users could chat with the chatbot. This added a more welcoming experience for users.

Finally, the chatbot was integrated into the Web Application using the Web Demo Integration within Dialogflow which consisted of adding a few lines of given code to a new component within the Web Application. Some text was also added to the component to explain to the user what the chatbot does and how to use it to ensure the level of intuitiveness within the application.

9.7.2 Deliverables

- AI chatbot
- Chatbot component in Web Application

9.7.3 Key Challenges and Accomplishments

The chatbot was delivered to incorporate artificial intelligence into the Web Application and integrated into the system via a component accessible by the Web Application nav bar.

The key challenge was fulfilling the user's request within the inline code editor in Dialogflow. The requests each had to wait for the last to finish to start or the data could not be brought back. To ensure this was the case, .then() was used so the next request would wait for the last and the correct data was brought back every time.

10. Testing

Two types of testing took place in the development of Swell Surf Check, the first being manual testing. This consisted of testing each of the requirements manually and ticking off the ones that passed. There was an overall pass rate of 91%. This can be seen in appendix 15.4. A failure corresponds to a requirement that was not included in the system.

The second type of testing that took place was automated testing within the Angular Web Application. When an Angular component is generated, a test file is generated along with it. Here, unit tests can be written around different aspects of the given component, for example to check if the component was created when it should have been, or a function is called when a button is clicked. These tests are written with the Jasmine framework and are then run using Karma. Karma uses a configuration file to set the start-up file, the testing framework, and the browser among other things.

Automated testing allows for easy refactoring as any changes to the code can be added without the worry of introducing new bugs, as well as adding new features without breaking the existing code.

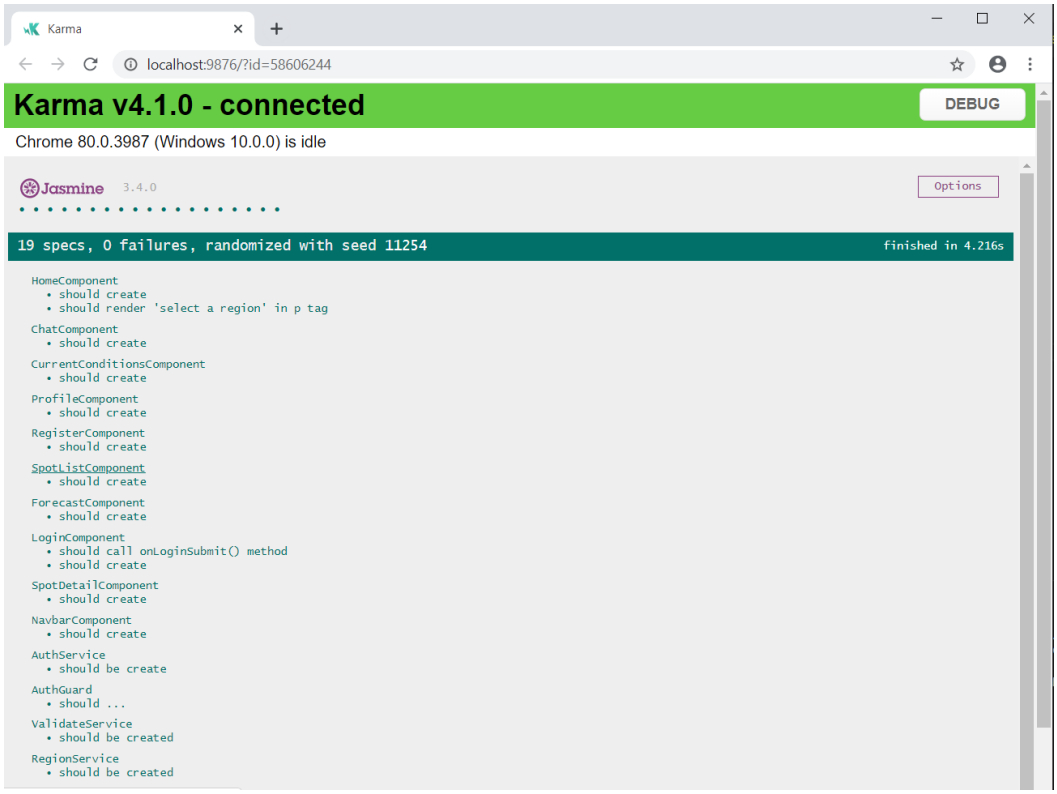


Figure 17 - Automated unit tests running with Karma

11. End Project Report

Swell Surf Check is a forecasting and information application which uses artificial intelligence to aid user searches. It allows for searches of surf spots around the South West and displays current and future surf conditions as well as information on the surrounding area.

Most of the project objectives have been fulfilled. The system allows the user to search for surf spots given a region or spot name. This takes the user to a spot detail page where the surf conditions are displayed along with a map and list of nearby points of interest and a more detailed surf report for experience surfers. There is also a working chatbot that users can ask about conditions for instant information.

Profile management is a feature that was not brought into the system, so users cannot change email addresses or passwords once they have signed up. Similarly, searching for a spot by region or string was only implemented in the Web Application. The Android Application has only a list the user can scroll through to find their desired surf spot.

Appendix 15.4 shows which of the initial project requirements have been fulfilled. 91% of the requirements were met, including all of the high priority requirements.

12. Project Post-Mortem

12.1 What Went Well?

The artificial intelligence aspect of the project went very well. While at the beginning of the project it was unclear how the chatbot would work and even if it would be

included, after conducting research and deciding to use a framework instead of writing it from scratch, it was fairly easy to implement it within the project. Dialogflow is well documented so writing the code that fulfilled the user intent was almost hassle free and with the included easy integration that comes with Dialogflow, integrating the chatbot itself into the Web Application required just a few lines of code which was much less than expected.

The testing of the user interface and the requirements testing also went well. Each user testing session ran smoothly with no problems from any of the users and valuable feedback was gained. From the feedback, new requirements were discovered that had not been previously thought of, as well as helping to tune the user interface to be more user friendly and intuitive in both the Web Application and the Android Application.

Adopting agile project management within the project was incredibly beneficial. Agile allowed the development to remain very flexible which was particularly important as the project was being run by a single developer. If a piece of work were not delivered in a sprint, it would simply be brought into the next sprint ready to be worked on. With relatively little development time, this ultimately aided the project being completed on time.

12.2 What Didn't Go Well?

Some aspects of the project did not go quite to plan. A few of the requirements were not met due to time constraints towards the end of the project, however they were a low priority, and therefore the priority system was used in the first place. Fortunately, all the high priority requirements were met to create a fully functioning final product.

At the beginning of the project it was hoped web hooks could be included to allow users to see changes in the surf conditions without having to refresh the page. It turned out however, that this could not be completed due to the use of AWS Lambdas. A downside to having a serverless application is that it is hard to integrate web sockets as there is there is no server, so it is hard for everything involved to know what everything else is doing. Amazon Web Services came up with a solution to this where a web socket API is created with a connect function and a disconnect function. The connect function points to a Lambda that, when called, inserts a document into the MongoDB database with a connection ID which is given when the web socket endpoint provided by the API is called from a particular device. The disconnect function removes the document from the database when the device is disconnected. An 'onMessage' custom function was then written which connects to a Lambda that gets all the connection IDs from the database and send the given message to all of them. This however did not work as the technology is new. A patch.js file was needed in some researched situations, but the code needed for this was unclear and most developers are yet to use AWS for web sockets so finding a solution to this problem proved to be too much. While this was disappointing, it was not high priority functionality.

```
PS D:\Uni\Year 3\soft 355\swellattempt10349302349\swell> wscat -c wss://skw636fovh.execute-api.eu-west-2.amazonaws.com/prod
Connected (press CTRL+C to quit)
> {"action" : "onMessage" , "message" : "Hello everyone"}
>
< {"message": "Forbidden", "connectionId":"FiORORdyLPECGbQ=", "requestId":"FiOb8GFcLPEFibg="}
```

Figure 18 - Attempting to call the onMessage function on the connected web socket API but getting a 'forbidden' message along with the connectionId.

12.3 What Would Be Performed Differently?

If the project were to be started again, some changes to the development process would be made. Firstly, the chatbot would have been implemented earlier on in the development cycle to allow it to be fine-tuned and other features be added. A prototype of the chatbot would also be factored into a sprint to include the functionality in a usability testing session to get feedback on what kinds questions users will ask to train the bot.

Work on development would also have started sooner, perhaps enough time for an extra sprint to work on improving the stability and performance of the applications. For example, the first time the Web Application is loaded in a while, the loading times are greater than normal. This is probably to do with Heroku, however, there were more pressing matters in the given time frame.

12.4 Future Enhancements

Future enhancements could include releasing the mobile application on iOS to be available to more people across more platforms. This would be relatively easy using Ionic; however, a MacBook is required for iOS development. Users not using an Android device can currently access Swell Surf Check on a mobile web browser, however.

The scope of the chatbot could be made larger so that users can ask many different types of questions. At present, users can only ask what the surf conditions are at a particular spot, however this could be changed so that users could ask where the conditions are best currently, or which spot suits their needs based on their skill level, for example. Also, currently the chatbot works only when it is running on Dialogflow, this is because it needs to be approved by Google before release and in the current situation with Coronavirus, a response regarding this from Google was unavailable.

In future, a filter system could also be implemented so that when a user searches for a spot, they can filter results, perhaps based on their current location, or the difficulty of the surf break, to decide where they want to go. There could also be user reviews or guides for each spot to develop a community between Swell Surf Check users.

13. Conclusions

This report has highlighted the need for a surf check application in a world where surfing is at its peak. Swell Surf Check is a forecast and information source that allows users to discover new surf breaks in the South West of England – one of the most popular surf destinations in the world. It is available to the majority of devices across the world, therefore has a huge potential user base.

Swell Surf Check was developed using modern technologies like Amazon Web Services Serverless technology, Angular, Node.js and MongoDB.

Swell Surf Check solves the problem of people needing to know surf conditions and other nearby amenities before they get to the beach. Users can search by region or by spot name if they know the spot they are going to and can view current and future surf conditions in a concise and easy to read manner. The use of an AI chatbot allows users to ask about surf conditions quickly when on the go in an exciting new way that is not offered anywhere else on the internet.

14. References

Ahmed, A., 2011. *Software Project Management.* Boca Raton, FL: Auerbach Publications.

Amazon, n.d. *Amazon.* [Online]
Available at: https://aws.amazon.com/lambda/
[Accessed 17 April 2020].

Android, 2020. *Android Developers.* [Online]
Available at: https://developer.android.com/docs/quality-guidelines/core-app-quality
[Accessed 19 April 2020].

Angular, 2020. *Angular.* [Online]
Available at: https://angular.io/docs
[Accessed 17 April 2020].

Atlassian, 2020. *Atlassian.* [Online]
Available at: https://www.atlassian.com/agile
[Accessed 19 April 2020].

Bootstrap, 2020. *Get Bootstrap.* [Online]
Available at: https://getbootstrap.com/
[Accessed 19 April 2020].

Caliman, D., 2015. *Creative Tim.* [Online]
Available at: https://www.creative-tim.com/blog/web-design/use-not-use-bootstrap-framework/
[Accessed 19 April 2020].

Churm, T., n.d. *Usability Geek.* [Online]
Available at: https://usabilitygeek.com/an-introduction-to-website-usability-testing/
[Accessed 25 April 2020].

Dam, R. F. & Teo, Y. S., 2020. *Interaction Design.* [Online]
Available at: https://www.interaction-design.org/literature/article/design-thinking-get-started-with-prototyping
[Accessed 24 April 2020].

Git, 2020. *Git.* [Online]
Available at: https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control
[Accessed 21 April 2020].

Google, 2020. *Google.* [Online]
Available at: https://developers.google.com/places/web-service/policies
[Accessed 20 April 2020].

gov.uk, 2018. *Gov.uk.* [Online]
Available at: http://www.legislation.gov.uk/ukpga/2018/12/contents/enacted
[Accessed 20 April 2020].

Haas, N., 2011. *Orbit Media.* [Online]
Available at: https://www.orbitmedia.com/blog/7-reasons-to-wireframe/
[Accessed 24 April 2020].

Harkushko, L., n.d. *Yalantis.* [Online]
Available at: https://yalantis.com/blog/when-to-use-angular/
[Accessed 17 April 2020].

Hickman, A., 2019. *Surf Holidays.* [Online]
Available at: https://www.surfholidays.com/blog/the-best-surf-forecasting-websites
[Accessed 19 April 2020].

Hirschberg, J. & Manning, C. D., 2015. Advances in natural language processing. *Science Mag,* 349(6245), pp. 261-266.

INKONIQ, 2017. *Medium.* [Online]
Available at: https://medium.com/inkoniq-blog/top-7-reasons-to-pick-ionic-framework-for-mobile-apps-30169275fb58
[Accessed 17 April 2020].

Leffingwell, T. G., 2017. *MDDI.* [Online]
Available at: https://www.mddionline.com/incorporating-stress-user-testing
[Accessed 20 April 2020].

Lipchain, 2018. *Lipchain.* [Online]
Available at: https://medium.com/lipchain/how-many-surfers-are-there-in-the-world-c43d04b93442
[Accessed 16 April 2020].

Magic Seaweed, 2020. *Magic Seaweed.* [Online]
Available at: https://magicseaweed.com/app/
[Accessed 19 April 2020].

MongoDB, n.d. *MongoDB.* [Online]
Available at: https://www.mongodb.com/what-is-mongodb
[Accessed 17 April 2020].

Rehan, A., 2019. *Geek Flare.* [Online]
Available at: https://geekflare.com/chatbot-development-frameworks/#Dialogflow
[Accessed 17 April 2020].

Schlicht, M., 2016. *Chat Bots Magazine.* [Online]
Available at: https://chatbotsmagazine.com/the-complete-beginner-s-guide-to-chatbots-8280b7b906ca
[Accessed 17 April 2020].

Spy Cloud, 2019. *Spy Cloud.* [Online]
Available at: https://spycloud.com/how-long-would-it-take-to-crack-your-password/
[Accessed 20 April 2020].

Trello, 2020. *Trello.* [Online]
Available at: https://help.trello.com/article/708-what-is-trello
[Accessed 19 April 2020].

Visit Britian, 2015. *Visit Britain.* [Online]
Available at: https://www.visitbritain.org/sites/default/files/vb-corporate/Documents-Library/documents/England-documents/south_west_2015.pdf
[Accessed 16 April 2020].

Visual Paradigm, n.d. *Visual Paradigm.* [Online]
Available at: https://www.visual-paradigm.com/scrum/why-fixed-length-of-sprints-in-scrum/
[Accessed 19 April 2020].

15. Appendices

15.1 User Guide

Minimum System Requirements

Web Application

Operating System: Any

RAM: 1GB

Software: Internet Browser

*Active internet connection required to use all Swell Surf Check services.

Android Application

Operating System: Android 4.2 (Jelly Bean)

RAM: 1GB

*Active internet connection required to use all Swell Surf Check services.

Installation

Web Application

Navigate to https://fast-chamber-85467.herokuapp.com/ from any browser.

Android Application

Open the project in Android Studio and run either on an emulator or connected device. There are plans to release the app to the Play Store in the future.

Overview

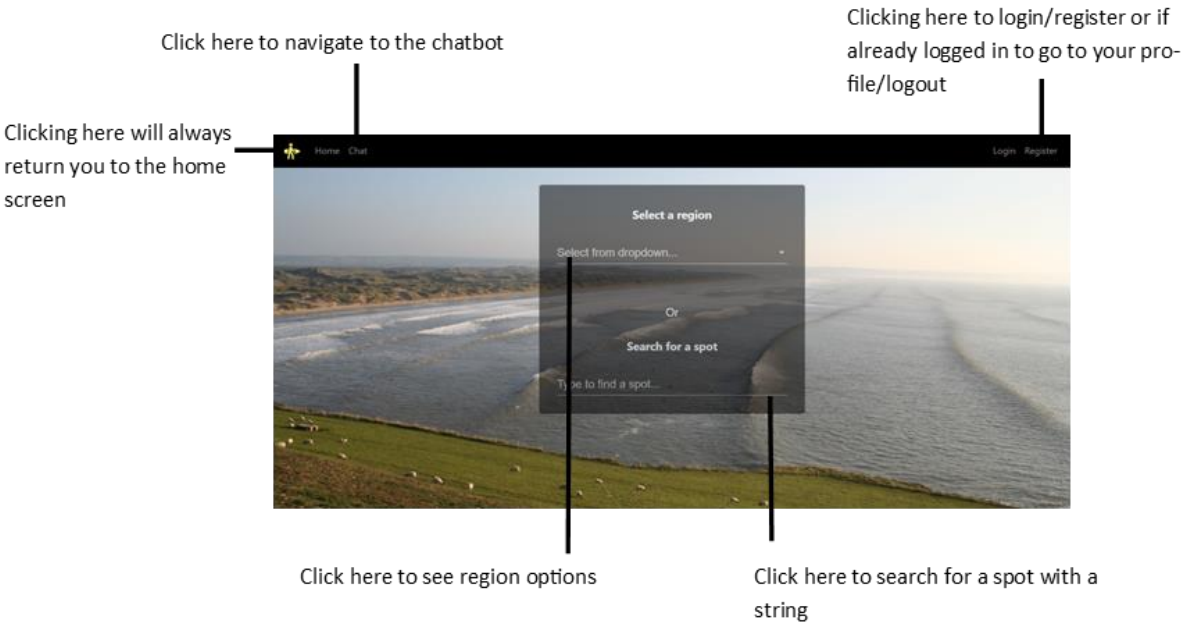
Swell Surf Check is a surf forecast and information provider that shows the current and future surf conditions of surf spots around the South West of England as well as nearby points of interest for each spot.

Swell Surf Check also allows users to check the conditions by asking an AI chatbot what the surf is doing at a given spot. This represents a new and exciting way of checking the surf.

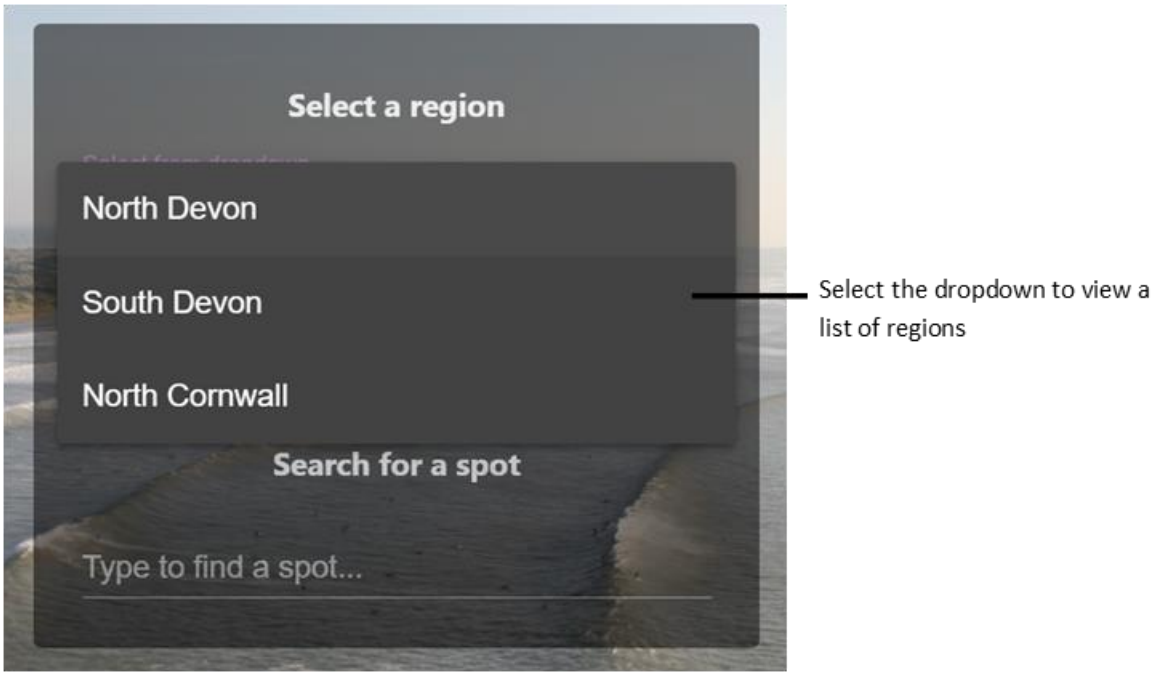
Guide

Web Application

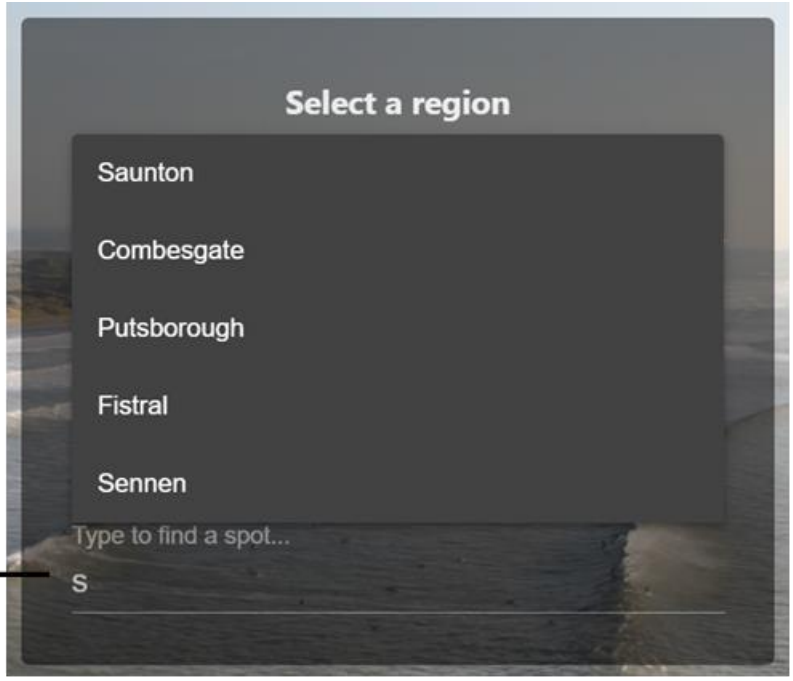
Home Screen



Region Search



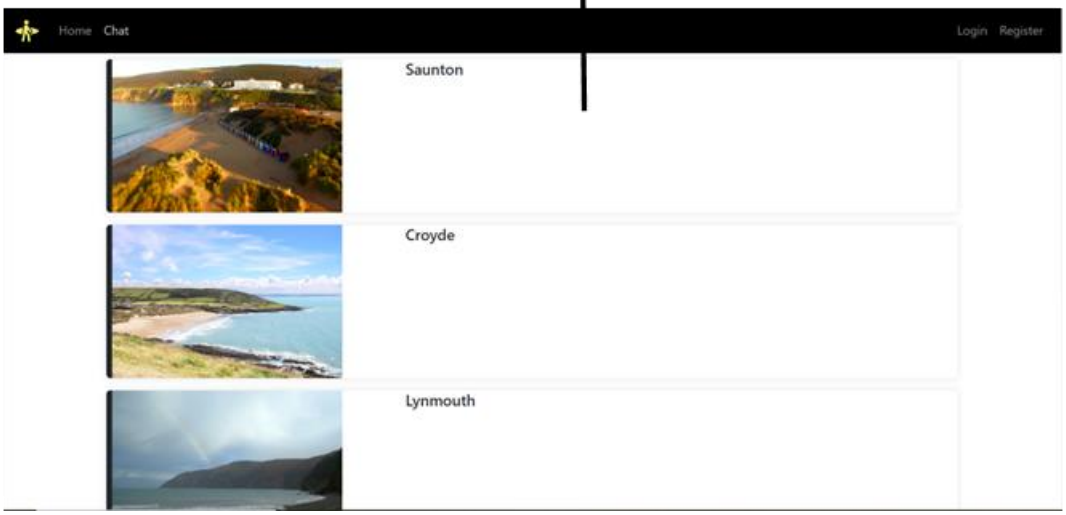
String Search



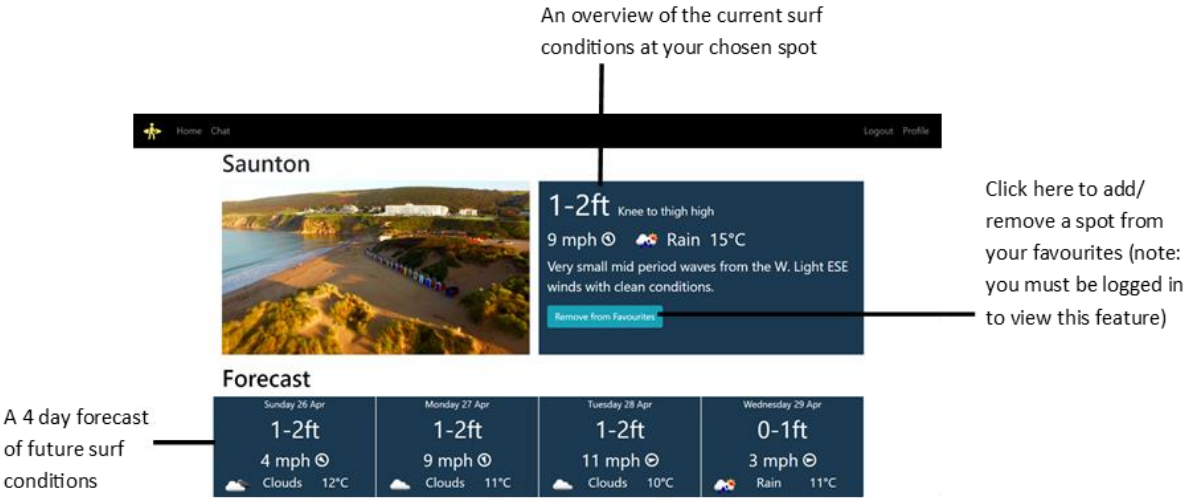
Type in the box to see a list of options based on your search

Spot List

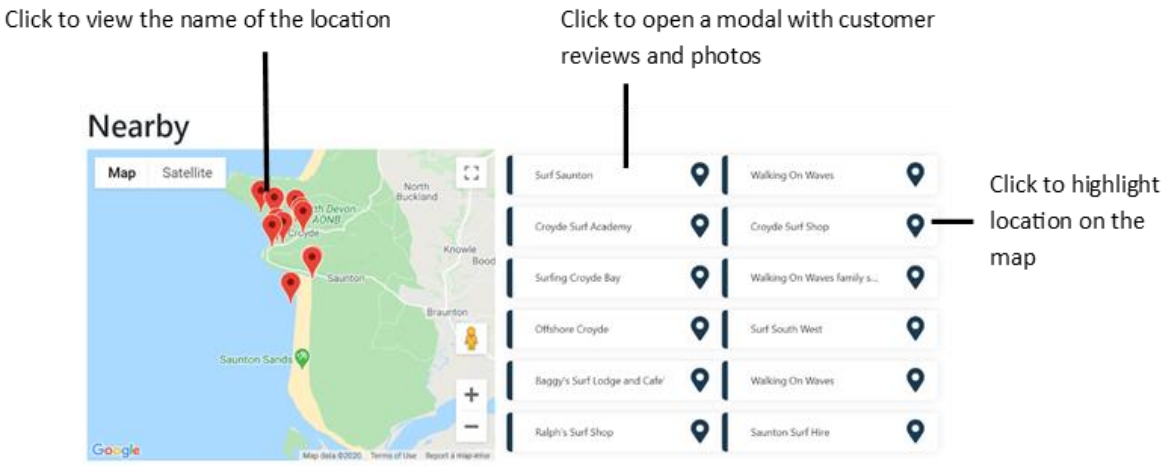
After selecting a region, click a spot from the spot list to view the forecast



Current Conditions and Forecast



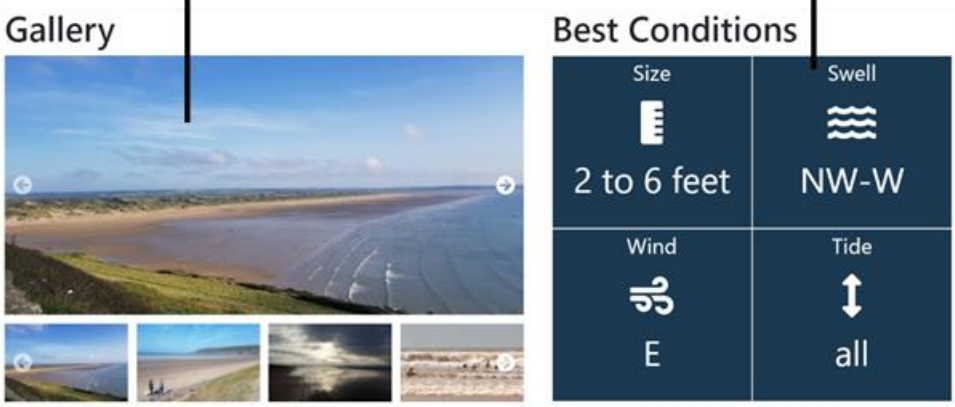
Nearby Points of Interest



Gallery

Click the arrows to navigate between pictures or click the picture for a full screen view

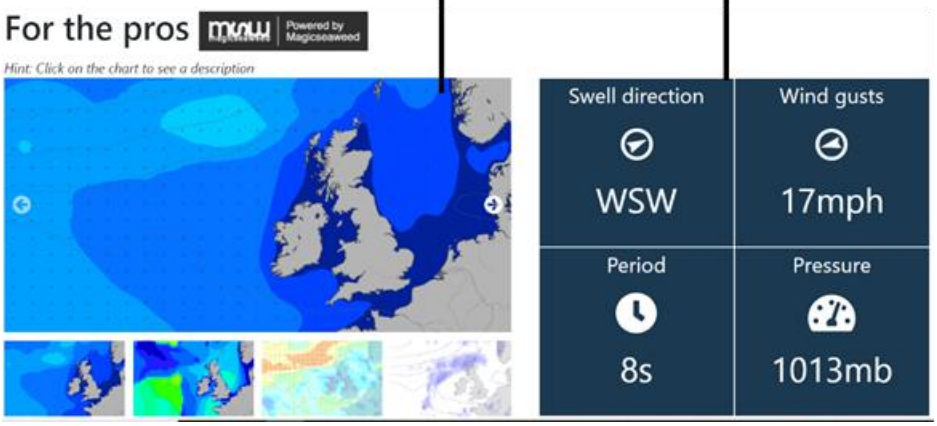
An overview of when the spot is at its best



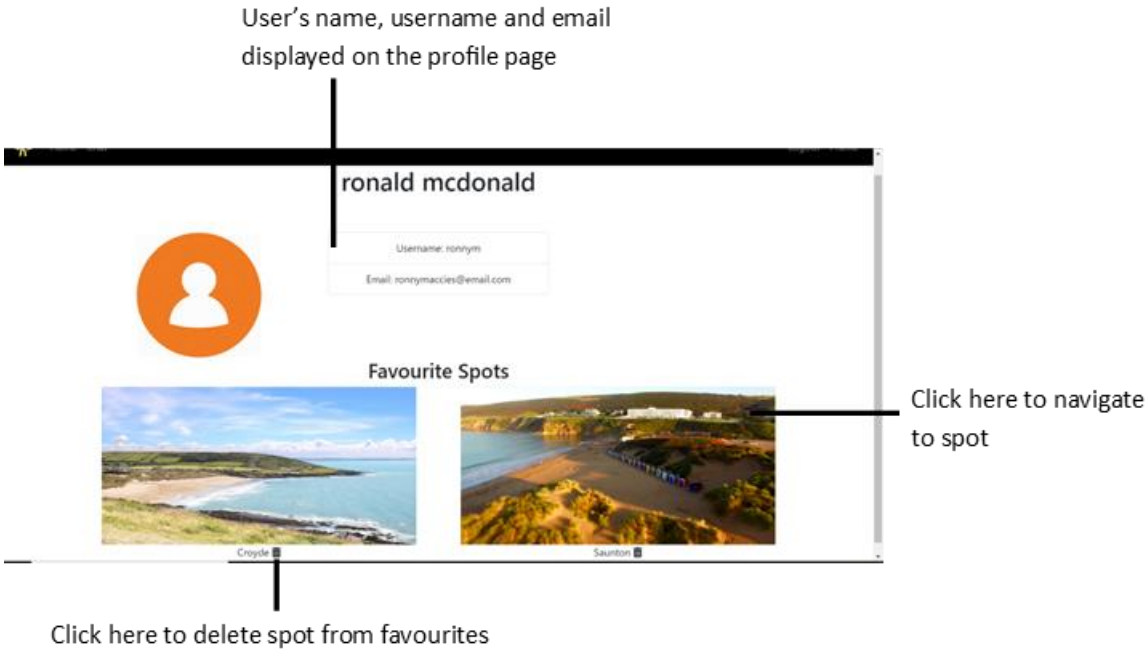
Advanced Forecast

Like the gallery, use the arrows to navigate the charts and click the chart for a full screen view and to view a description of what the chart depicts

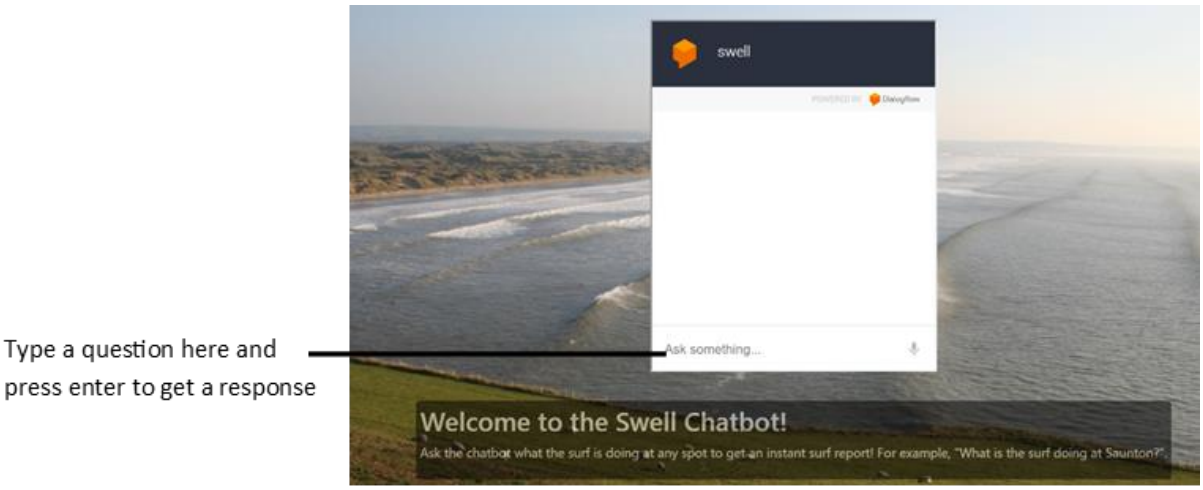
Current conditions information for more experience surfers



Profile

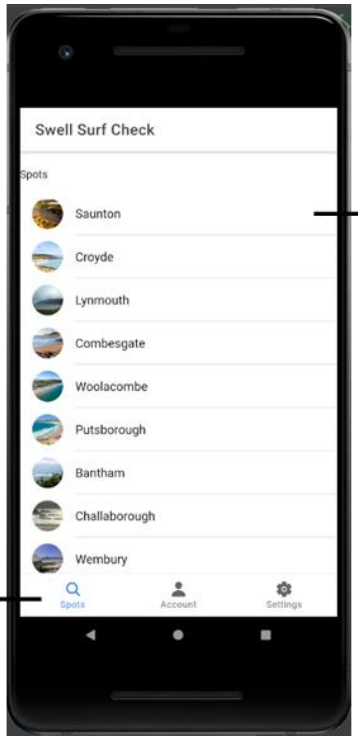


Chatbot



Android Application

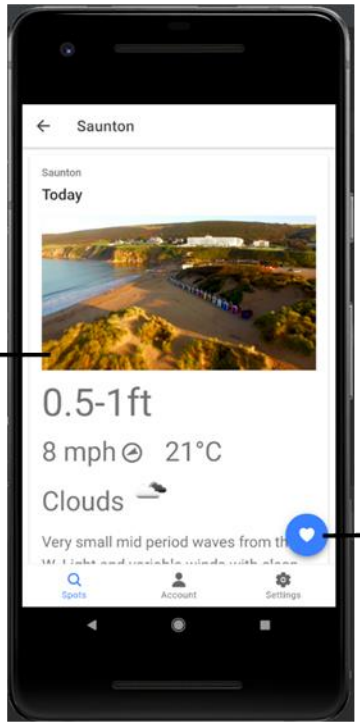
Home



Scroll to browse spots and tap to navigate to the spot's detail page

Tap on the spots tab to navigate to the spots page

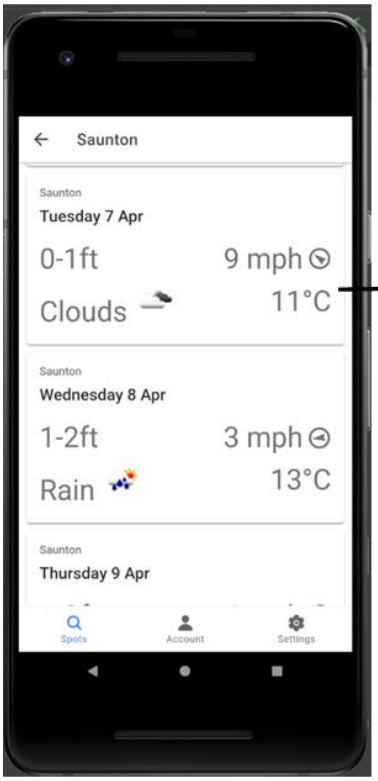
Current Conditions



The top of the spot detail page shows the current surf and weather conditions

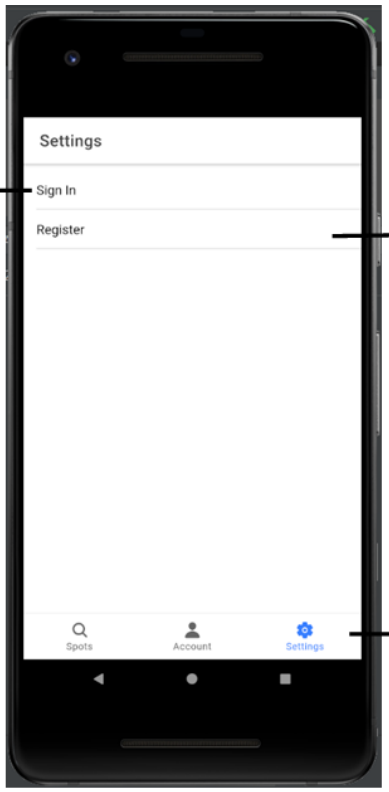
If logged in, tap here to add spot to favourites

Forecast



Scroll on the spot detail page to view a 4 day forecast

Settings

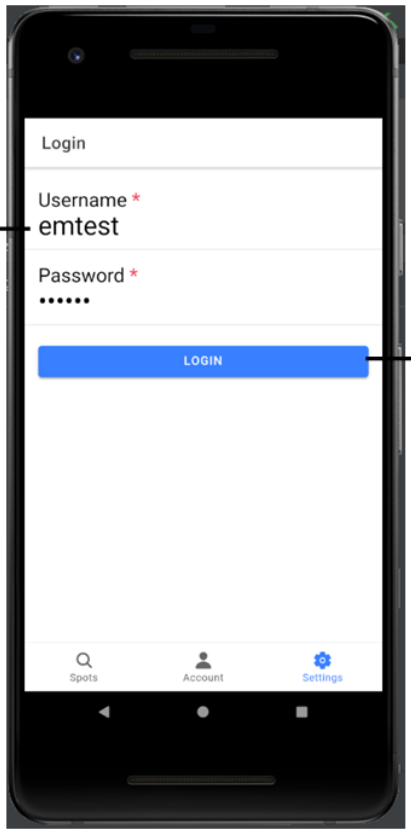


Tap to sign in if you already have an account

Tap to register if you don't already have an account

Tap the settings tab to navigate to settings page

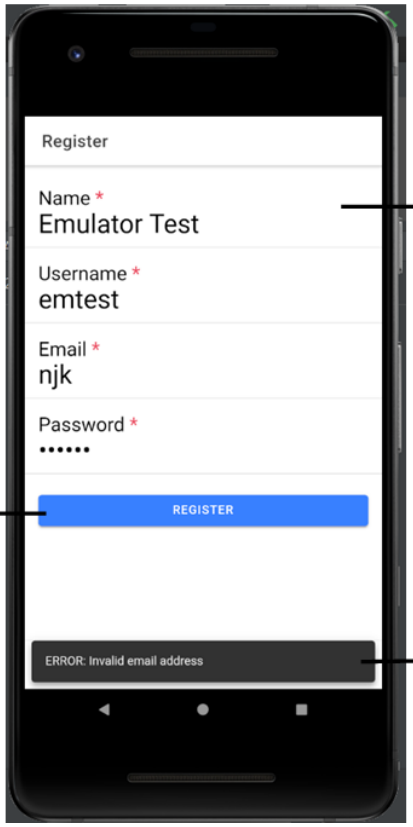
Login



Enter the credentials you used to register to login to the system

Tap to login

Register

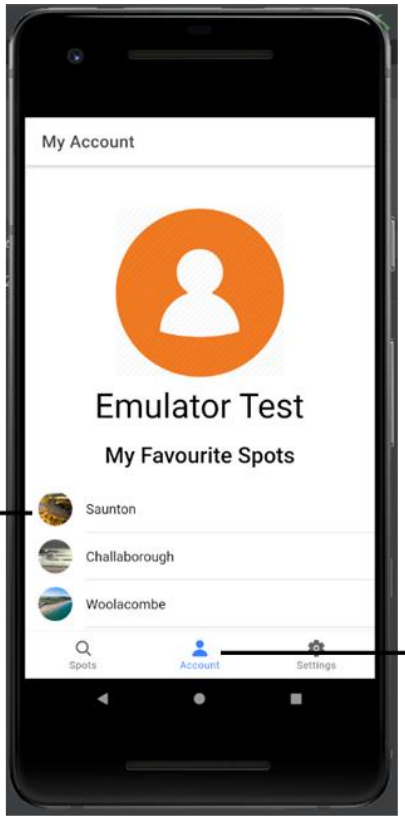


If not already registered navigate to the register page and choose credentials

Tap to register

If any fields are blank or an invalid email is used, an error message will be shown

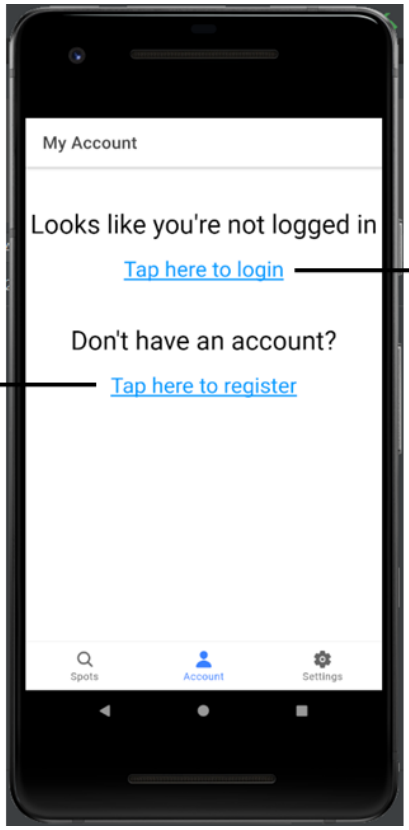
Account Logged In



If logged in, tap to go to your favourite spots

Tap the account tab to navigate to the account page

Account Not Logged In



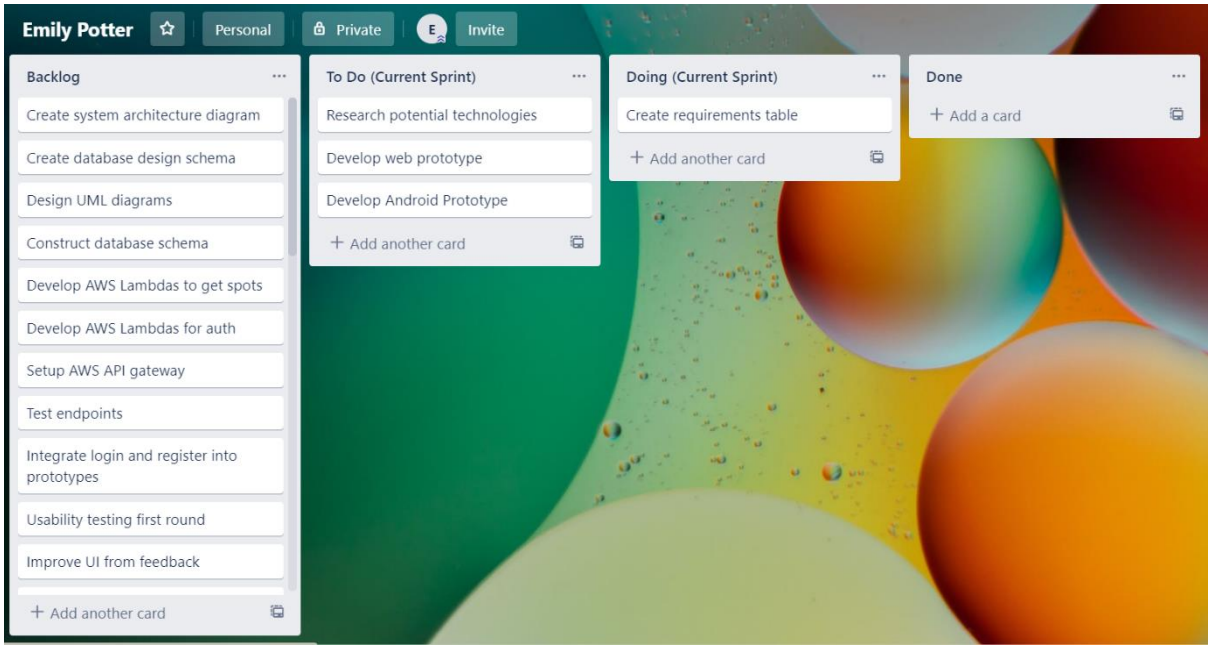
If you are not logged in on the account page tap here to login if you have already registered

Tap here to register if you don't yet have an account

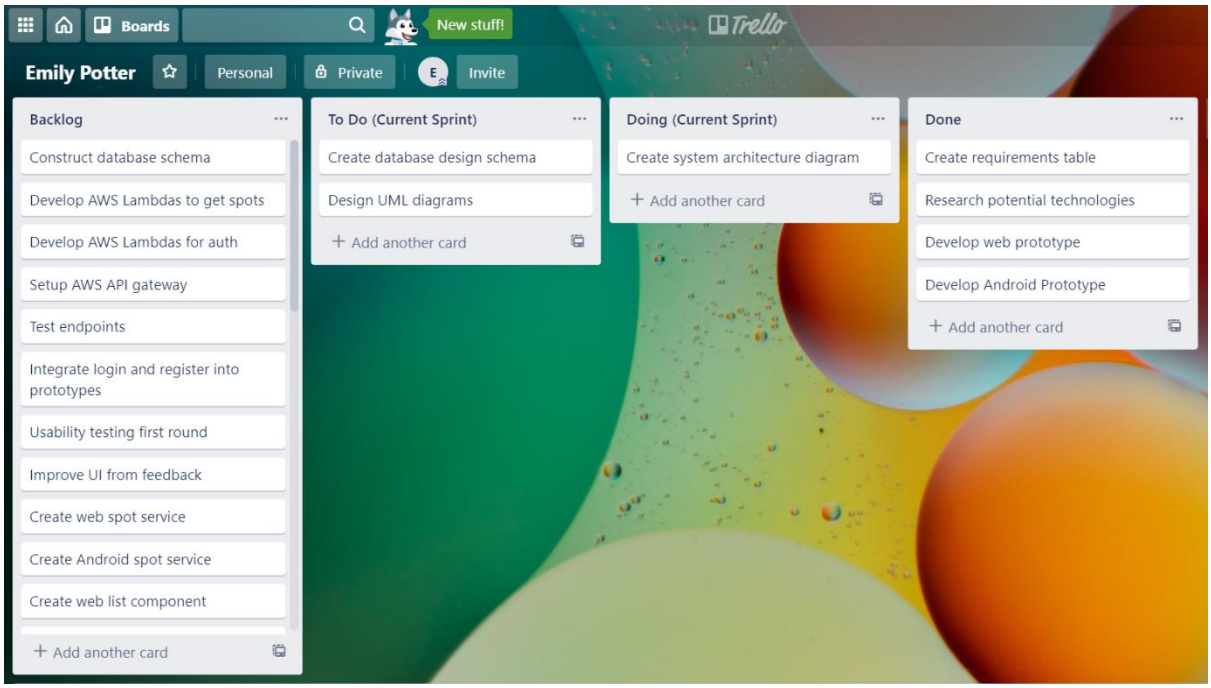
15.2 Project Management

15.2.1 Trello Boards

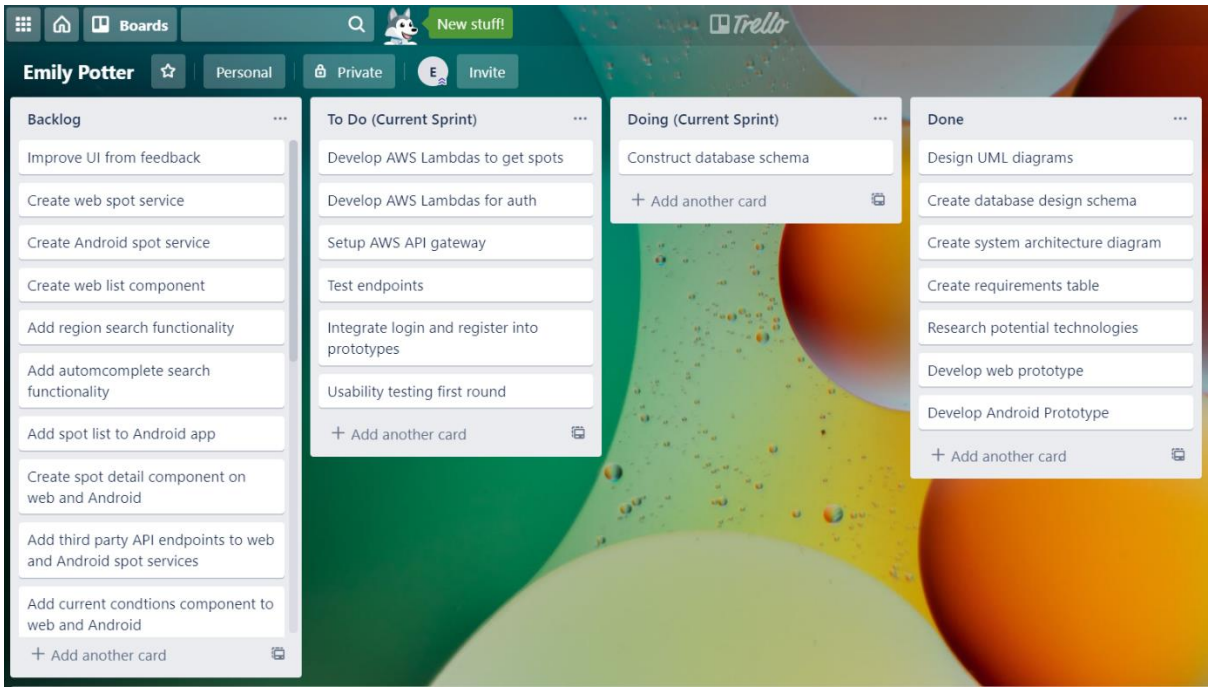
Trello 1



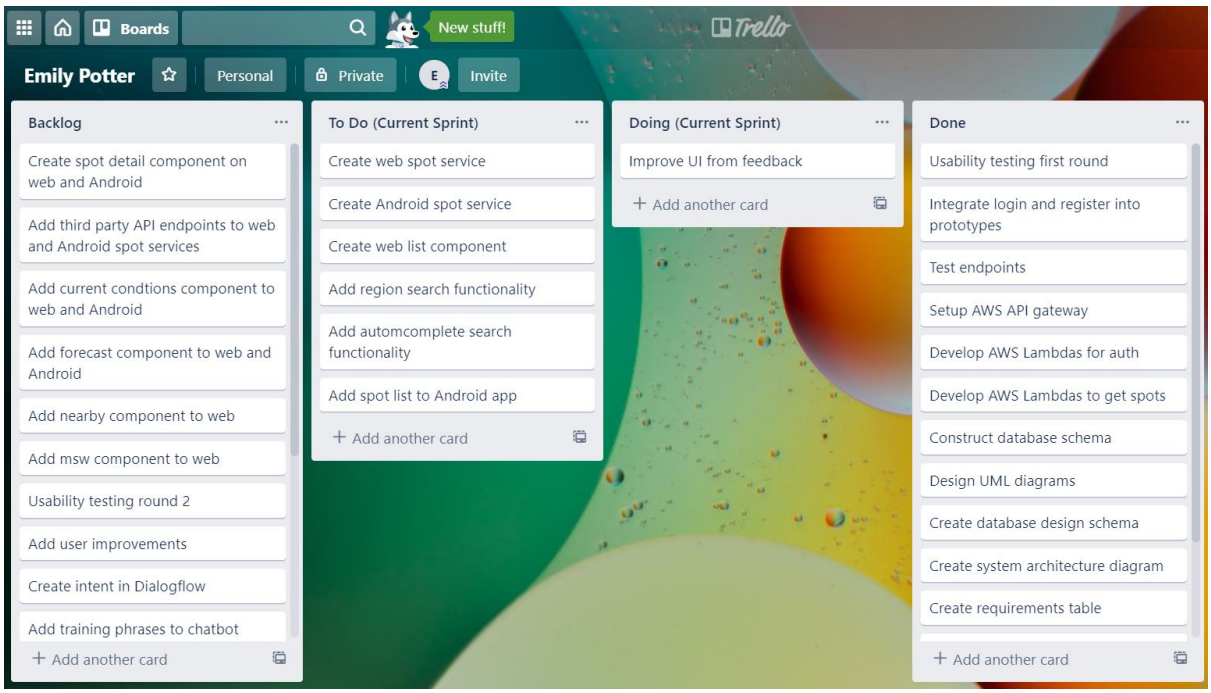
Trello 2



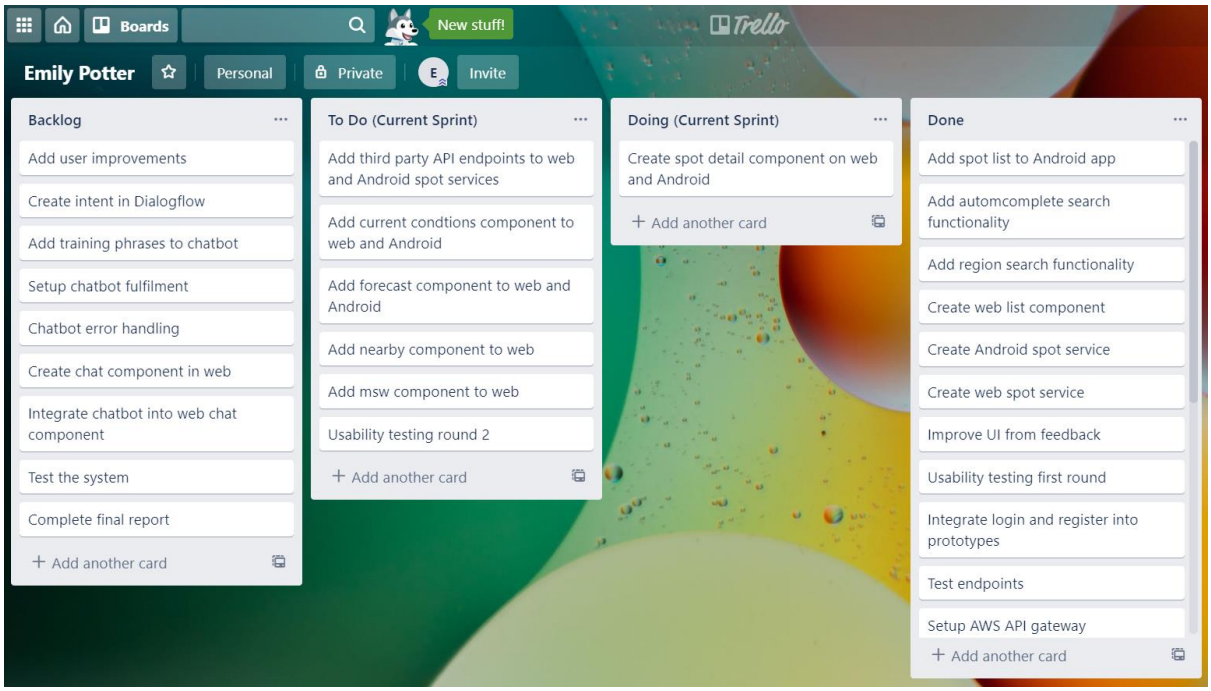
Trello 3



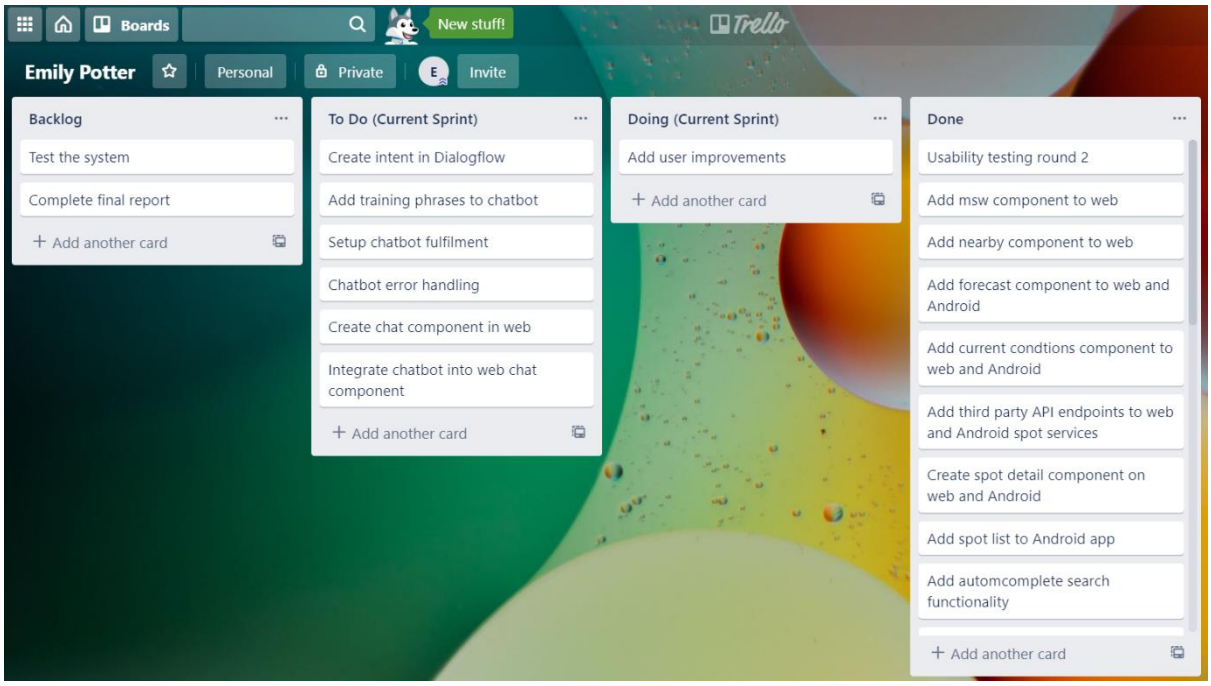
Trello 4



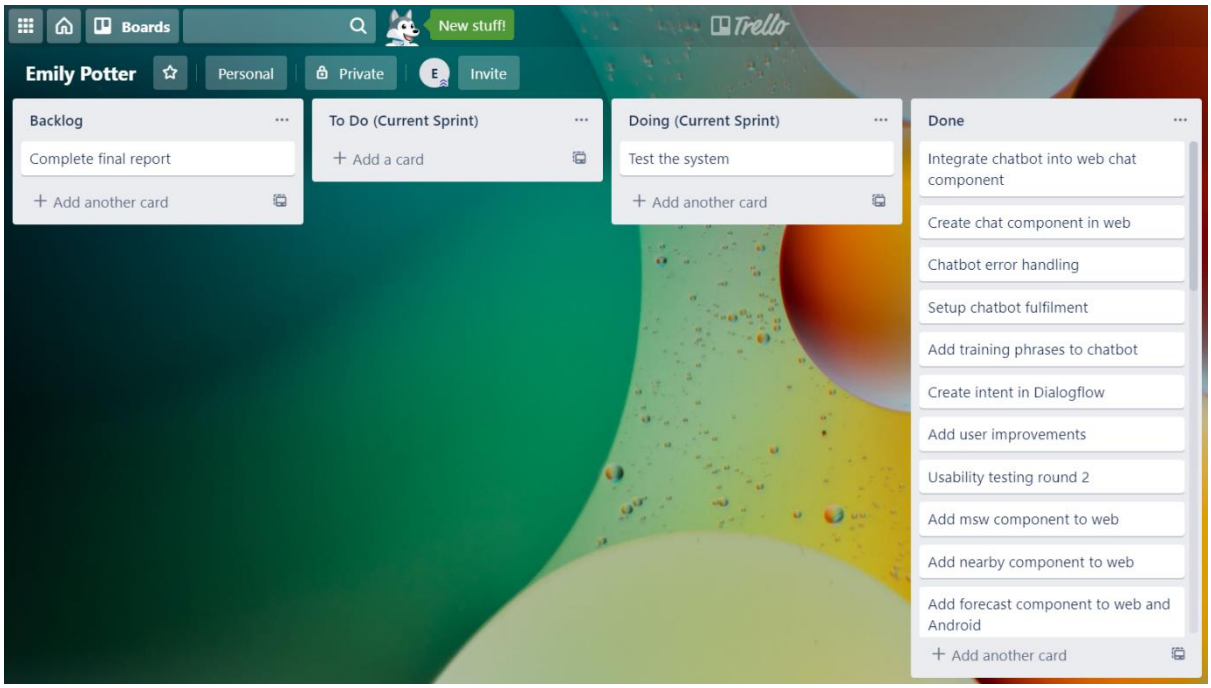
Trello 5



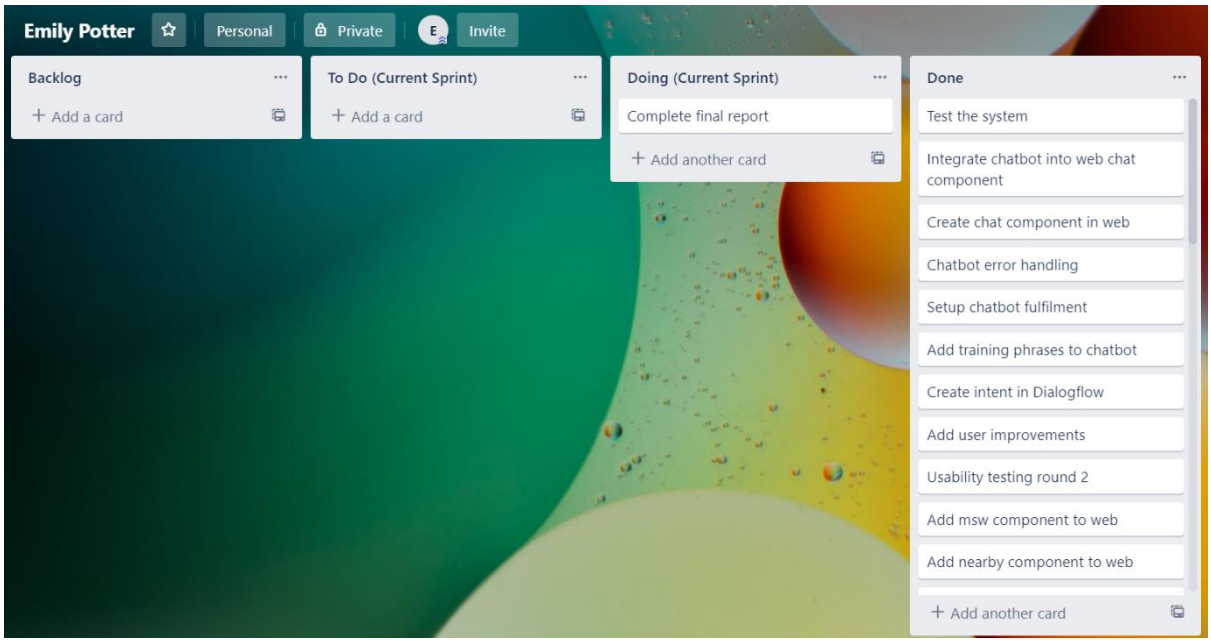
Trello 6



Trello 7



Trello 8



15.2.2 Requirements

Functional requirements

Requirement	Priority
The user should be able to login with a button on the nav bar in the Web Application and a button in the settings of the Android Application.	2
The user should be able to register an account with a chosen username and password.	2
The user should be able to see their account details on a profile page.	2
The user should be presented with a forgotten password link in the case they forget their login credentials.	4
The user should be presented with a toast if the credentials they enter are wrong.	2
The user should be able to change their password.	4
The user should be presented with an error message if they try to register with invalid credentials.	2
The user should be presented with an error message if the username they try to register with is already taken.	3
The user should be presented with a home screen that they can search and select surf spots from.	1
When the user clicks the Swell Surf Check icon in the Web Application, they should be redirected back to the home screen.	2
In the Web Application, the user should be able to search for a spot with the aid of autocomplete.	2
The user should be presented with a spot detail page after choosing a spot.	1
The spot detail page should include the current surf conditions.	1
The spot detail page should include a forecast of future surf conditions.	1
The spot detail page should include nearby points of interest.	2
The nearby places of interest should be placed on a map.	2
When clicking on a point of interest, the user should be presented with a modal with more information.	2
The information modal should include a gallery and reviews.	3
The spot detail page should include what conditions for each spot is best.	3

Magic Seaweed's API should be integrated to show charts and other advanced information.	3
The user should be able to ask a chat bot what the conditions are doing at a spot.	2
The user should be able to ask a chat bot which spot has the best conditions around them.	4
The user should be easily able to navigate between the chat bot, surf forecasts and profile pages easily.	1
The user should be able to add a spot to their favourites for easy access from their profile page.	3
The user should be able to remove spots from their favourites.	4
The UI should be intuitive.	1
The user should not be able to access the profile page unless they are logged in.	1

User requirements

Requirement	Priority
The user will be able to register for an account.	2
The user will be able to login to the Swell Surf Check on both the Web Application and the Mobile Application.	2
The user will be able to view their profile on a profile page.	2
The user will be able to search for a spot from the home screen.	1
The user will be able to browse spots based on a region.	1
The user will be able to search for a spot with a user entered search string.	1
The user will be able to select a result based on their search string from an autocomplete dropdown.	2
The user will be able to see spots based on their search in the form of a list.	1
The user will be able to view the current surf conditions of a spot.	1
The user will be able to see the future surf conditions of a spot.	1
The user will be able to see nearby points of interest in the form of a list.	2
The user will be able to see nearby points of interest on a map.	3
The user will be able to add a spot to their favourites when logged in.	2
The user will be able to delete a spot from their favourites when logged in.	4

The user will be able to see their favourite spots from the profile page when logged in.	2
The user will be able to ask a chatbot what the surf conditions are doing at a spot.	1
The user will be able to view the best conditions for a spot.	2
The user will be able to see advanced surf forecast information.	3
The user will be able to see advanced charts that predict different aspects of the surf.	3

Non-functional requirements

Requirement	Priority
There should be a backup of the source code.	1
The usability testing should say the applications are easy to use.	1
The applications should be accessible to those with disabilities.	2
The system should be testable via automated testing.	1
Response times for user interaction should not exceed 5 seconds.	1
The code should be easily modifiable and adhere to known coding practices.	1
The system should be secure from hackers, e.g. by hashing and salting passwords.	1
There should be documentation for the application.	2

15.2.3 Stage Plans

Stage 1

Task	Expected start date	Expected end date	Deliverables/Outcomes
Specification of requirements	20/01/20	20/01/20	Requirements table
Evaluation of possible technologies	21/01/20	21/01/20	Technologies decided upon
Develop simple prototypes	22/01/20	24/01/20	Web and Android Prototypes

Stage 2

Task	Expected start date	Expected end date	Deliverables/Outcomes
Design system architecture	25/01/20	25/01/20	System architecture diagram

Design database schema	26/01/20	26/01/20	Database schema documentation
Design UML diagrams	27/01/20	27/01/20	UML diagrams

Stage 3

Task	Expected start date	Expected end date	Deliverables/Outcomes
Construct database schema	28/01/20	30/01/20	Functioning database
Develop AWS Lambdas to get spots	31/01/20	01/02/20	AWS Lambdas for spots
Develop AWS Lambdas for API authentication	02/02/20	03/02/20	AWS Lambdas for auth
Setup AWS API Gateway to provide endpoints for Lambdas	04/02/20	05/02/20	API endpoints
Test endpoints	06/02/20	06/02/20	Ensure login, register, and get spots endpoints work in Postman
Integrate login and register functionality on prototypes	07/02/20	07/02/20	Working login and register forms
Get users to test functionality	08/02/20	10/02/20	Usability feedback

Stage 4

Task	Expected start date	Expected end date	Deliverables/Outcomes
Adjust user interface of prototypes given user feedback from previous sprint	11/02/20	12/02/20	Improved UI
Create a spot service in the Angular Web App that calls the AWS endpoints that get spots	13/02/20	14/02/20	Spot service for web
Create a spot service in the Ionic Android App that calls the AWS endpoints that get spots	15/02/20	16/02/20	Spot service for Android

Create spot list component to display search results in web	17/02/20	18/02/20	Web spot list
Add region search functionality in web	19/02/20	20/02/20	Region search in web leading to spot list
Add autocomplete search for spots in web	21/02/20	23/02/20	Autocomplete search for spots in web
Add spot list to home screen of mobile	24/02/20	25/02/20	Android spot list

Stage 5

Task	Expected start date	Expected end date	Deliverables/Outcomes
Create spot detail component on web	26/02/20	26/02/20	Web spot detail component
Create spot detail component on Android	27/02/20	27/02/20	Android spot detail component
Configure spot service to get data from Surfline API, weather API and Google Places API in web and Android	28/02/20	29/02/20	Methods to get data in front end
Add current conditions component to web spot detail component with data from Surfline API	01/03/20	03/03/20	Current conditions component for web
Add current conditions component to Android spot detail component with data from Surfline API	04/03/20	05/03/20	Current conditions component for Android
Add forecast component to web spot detail component with data from weather API	06/03/20	08/03/20	Forecast component for web
Add forecast component to Android spot detail component with data from Surfline API	09/03/20	11/03/20	Forecast component for Android

Add nearby component to web spot detail component with data from Google Places API	12/03/20	14/03/20	Nearby component
Add For the Pros component to web spot detail component with data from Magic Seaweed API	15/03/20	16/03/20	For the Pros component
Get users to test new functionality	17/03/20	18/03/20	Usability feedback

Stage 6

Task	Expected start date	Expected end date	Deliverables/Outcomes
Create check surf intent in Dialogflow	19/03/20	21/03/20	Check surf intent
Add training phrases to Dialogflow	22/03/20	23/03/20	Bot training
Setup webhook fulfilment in Dialogflow to fulfil user's request for data	24/03/20	26/03/20	Firebase function to check surf at user specified location
Error handling	27/03/20	28/03/20	Users able to type anything without breaking chatbot
Create chat component in web application	29/03/20	29/03/20	Web chat component
Integrate chatbot into web chat component	30/03/20	02/04/20	Working chatbot in web application

Stage 7

Task	Expected start date	Expected end date	Deliverables/Outcomes
Test the system	03/04/20	10/04/20	Requirements checklist

Stage 8

Task	Expected start date	Expected end date	Deliverables/Outcomes
Complete final report	10/04/20	13/05/20	Final report

15.2.4 Sprint Reviews

Sprint Review

Sprint: 1

Date: 10/02/2020

Completed

- The database was created from the database schema created previously.
- AWS Lambdas were configured to get and post data to and from the database.
- AWS API Gateway was configured to produce endpoints to Lambdas.
- Login and register were implemented on prototypes, beginning to turn them into real systems.
- User feedback gained.

Not completed

All tasks for this sprint were completed.

What went well?

Setting up the database was easier than expected as it does not require any normalisation like a traditional SQL database. Once AWS was configured, calling the endpoints in the front-end applications was very straightforward.

What did not go well?

Connecting to the MongoDB from the Lambdas took more time than expected. This is thought to be because the technology is still quite new and there is not a lot of documentation around it yet. Furthermore, after connecting to the database and getting the data, it was found the response had to be set out in a particular way or a “malformed lambda response” error message would be returned. Therefore, the response was setup in the way that AWS require.

Sprint Review

Sprint: 2

Date: 25/02/2020

Completed

- User feedback from sprint 1 was integrated into the applications.
- Spot services were created in each application to call the endpoints created in sprint 1.
- Spot list component was created in each application.
- Region and autocomplete search were added to the web application.

Not completed

All tasks for this sprint were completed.

What went well?

It was much easier to design the UI once user feedback had been given so the system easily became more intuitive. Also, creating the spot services to get data from the database proved to be a straightforward task in both the Angular application and the Ionic application. The region search also was achieved without problems once the data was retrieved from the database.

What did not go well?

The autocomplete search function took more time to complete than expected. It was decided Material Design's autocomplete feature would be used; however, it was not known how the logic behind this worked so significant research had to be carried out. Once the logic was worked out the sprint was back on track.

Sprint Review

Sprint: 3

Date: 18/03/2020

Completed

- Spot detail components were added to each application.
- Spot services were configured to get data from third-party APIs.
- Current conditions component was added to each application with relevant data being displayed.
- Forecast component was added to each application with relevant data being displayed.
- Nearby component was added to the web application to display nearby points of interest in a list and on a map.
- User feedback gained.

Not completed

The "For the Pros" component was not completed so will be brought into the next sprint.

What went well?

Getting the data needed and displaying it to the user was straightforward with everything going to plan.

What did not go well?

While displaying the data to the user was easy, following the wire frame designs proved to be more difficult as sometimes CSS can be tricky. To combat this, flex box and Bootstrap's grid system were used to display the data in a logical and user-friendly way while also being responsive.

Sprint Review

Sprint: 4

Date: 02/04/2020

Completed

- “For the Pro’s” component from last sprint.
- Dialogflow intents and training phrases.
- Dialogflow webhook fulfilment.
- Integration of chatbot into web application.

Not completed

All tasks for this sprint were completed.

What went well?

Setting up the chatbot proved to be much easier than expected with Dialogflow being very well documented. Training phrases were added to an intent in a matter of minutes. Error handling within the chatbot was also easy to implement with Dialogflow automatically simply asking a user to ask something else if it does not know what the user is asking. A small talk agent was also easily installed into the chatbot allowing users to actually chat.

What did not go well?

Fulfilling a user’s request from the chatbot was more difficult. The in-line code editor was used with events being logged in Firebase, however, how to use this was unclear at the start. Once more research had been carried out though, the logic was worked out and the webhook was getting the data required and sending it to the user.

15.3 UML Diagrams

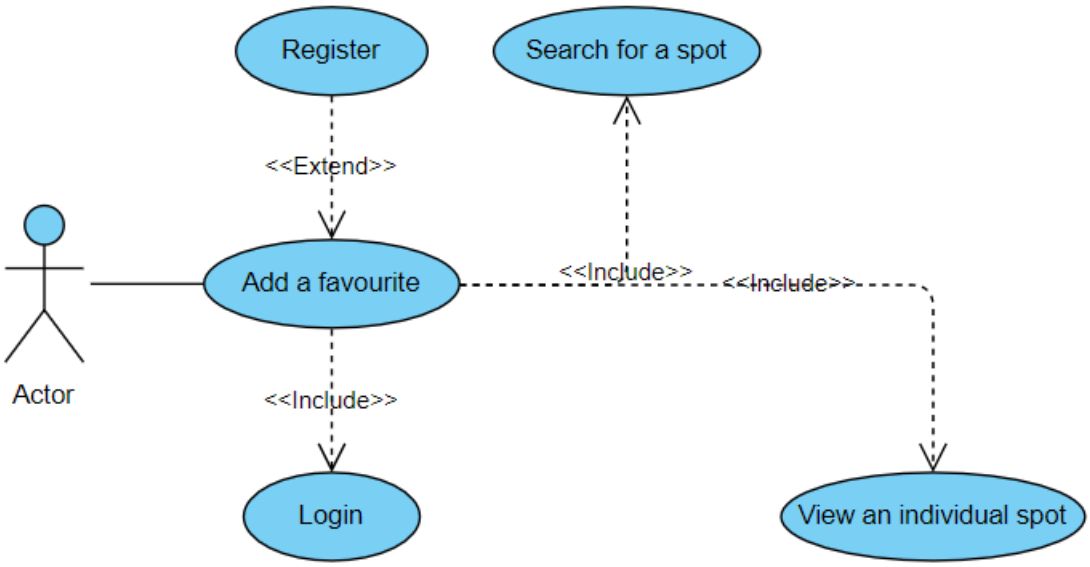


Figure 19 - Add spot to favourites use case

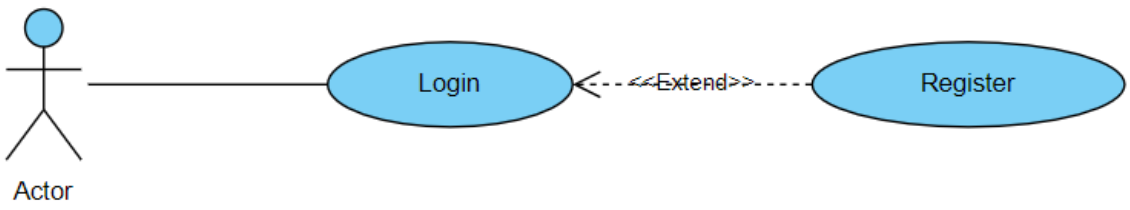


Figure 20 - Login use case

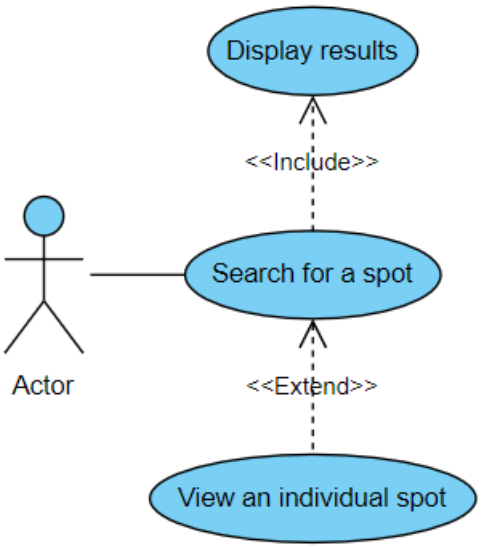


Figure 21 - Search for a spot user case

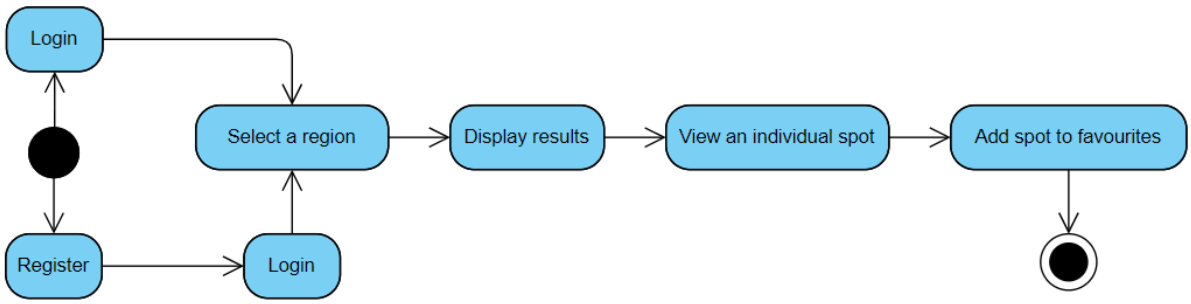


Figure 22 - Add to favourites activity diagram

15.4 Testing

Functional Requirements

Requirement	Priority	Completed?
The user should be able to login with a button on the nav bar in the Web Application and	2	Yes

a button in the settings of the Android Application.		
The user should be able to register an account with a chosen username and password.	2	Yes
The user should be able to see their account details on a profile page.	2	Yes
The user should be presented with a forgotten password link in the case they forget their login credentials.	4	No
The user should be presented with a toast if the credentials they enter are wrong.	2	Yes
The user should be able to change their password.	4	No
The user should be presented with an error message if they try to register with invalid credentials.	2	Yes
The user should be presented with an error message if the username they try to register with is already taken.	3	No
The user should be presented with a home screen that they can search and select surf spots from.	1	Yes
When the user clicks the Swell Surf Check icon in the Web Application, they should be redirected back to the home screen.	2	Yes
In the Web Application, the user should be able to search for a spot with the aid of autocomplete.	2	Yes
The user should be presented with a spot detail page after choosing a spot.	1	Yes
The spot detail page should include the current surf conditions.	1	Yes
The spot detail page should include a forecast of future surf conditions.	1	Yes
The spot detail page should include nearby points of interest.	2	Yes
The nearby places of interest should be placed on a map.	2	Yes

When clicking on a point of interest, the user should be presented with a modal with more information.	2	Yes
The information modal should include a gallery and reviews.	3	Yes
The spot detail page should include what conditions for each spot is best.	3	Yes
Magic Seaweed's API should be integrated to show charts and other advanced information.	3	Yes
The user should be able to ask a chat bot what the conditions are doing at a spot.	2	Yes
The user should be able to ask a chat bot which spot has the best conditions around them.	4	No
The user should be easily able to navigate between the chat bot, surf forecasts and profile pages easily.	1	Yes
The user should be able to add a spot to their favourites for easy access from their profile page.	3	Yes
The user should be able to remove spots from their favourites.	4	Yes
The UI should be intuitive.	1	Yes
The user should not be able to access the profile page unless they are logged in.	1	Yes

User Requirements

Requirement	Priority	Completed?
The user will be able to register for an account.	2	Yes
The user will be able to login to the Swell Surf Check on both the Web Application and the Mobile Application.	2	Yes
The user will be able to view their profile on a profile page.	2	Yes
The user will be able to search for a spot from the home screen.	1	Yes
The user will be able to browse spots based on a region.	1	Yes

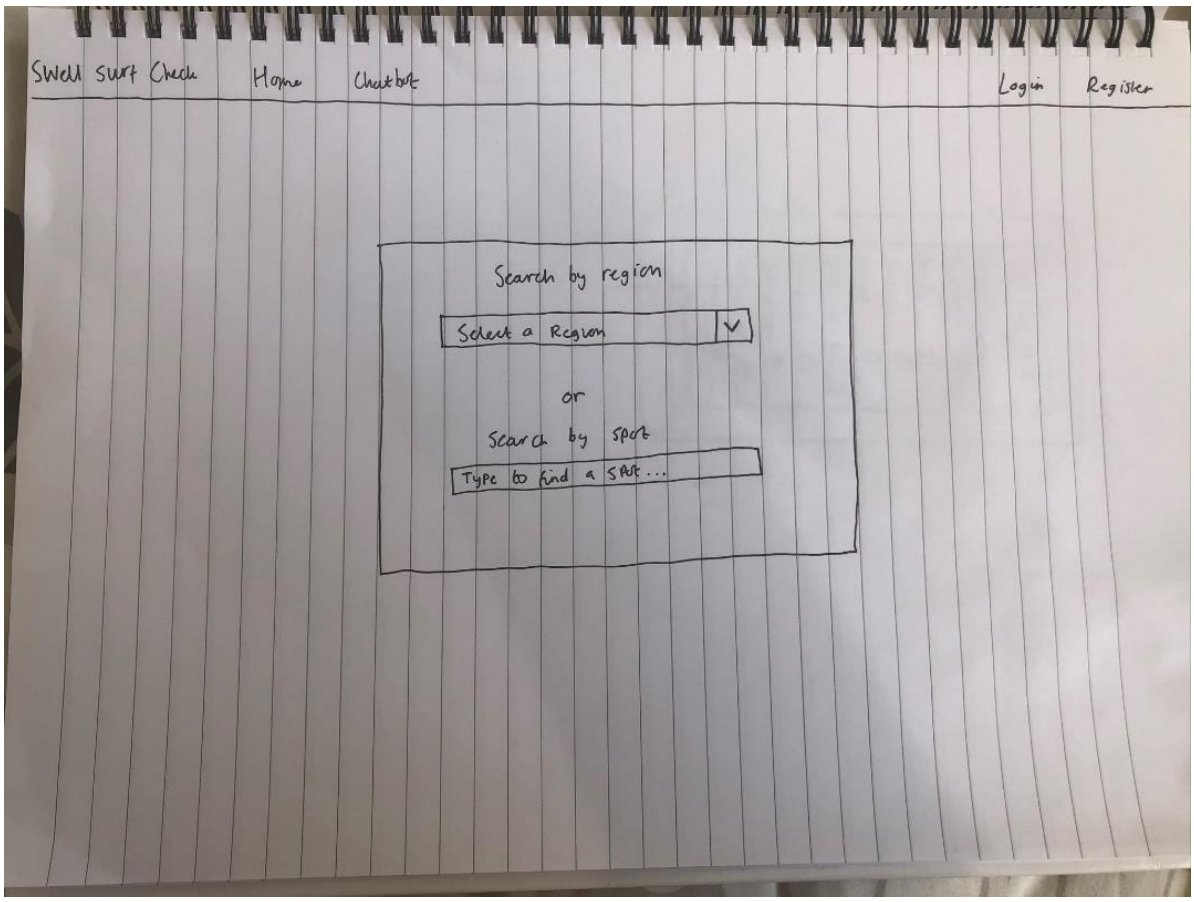
The user will be able to search for a spot with a user entered search string.	1	Yes
The user will be able to select a result based on their search string from an autocomplete dropdown.	2	Yes
The user will be able to see spots based on their search in the form of a list.	1	Yes
The user will be able to view the current surf conditions of a spot.	1	Yes
The user will be able to see the future surf conditions of a spot.	1	Yes
The user will be able to see nearby points of interest in the form of a list.	2	Yes
The user will be able to see nearby points of interest on a map.	3	Yes
The user will be able to add a spot to their favourites when logged in.	2	Yes
The user will be able to delete a spot from their favourites when logged in.	4	Yes
The user will be able to see their favourite spots from the profile page when logged in.	2	Yes
The user will be able to ask a chatbot what the surf conditions are doing at a spot.	1	Yes
The user will be able to view the best conditions for a spot.	2	Yes
The user will be able to see advanced surf forecast information.	3	Yes
The user will be able to see advanced charts that predict different aspects of the surf.	3	Yes

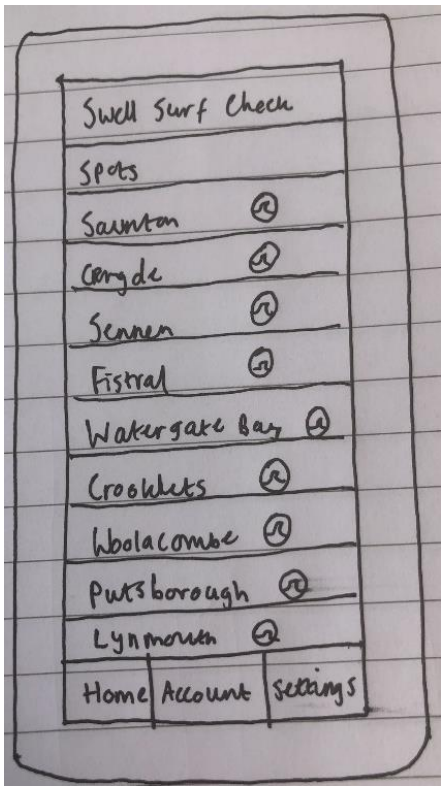
Non-Functional Requirements

Requirement	Priority	Completed?
There should be a backup of the source code.	1	Yes
The usability testing should say the applications are easy to use.	1	Yes

The applications should be accessible to those with disabilities.	2	No
The system should be testable via automated testing.	1	Yes
Response times for user interaction should not exceed 5 seconds.	1	Yes
The code should be easily modifiable and adhere to known coding practices.	1	Yes
The system should be secure from hackers, e.g. by hashing and salting passwords.	1	Yes
There should be documentation for the application.	2	Yes

15.5 Wire Frame Diagrams




Swell Surf Check
Spots
Saunton
Croyde
Sennen
Fistral
Watergate Bay
Croohlets
Woolacombe
Putsborough
Lynmouth
Home
Account
Settings

15.6 Questionnaire Results

Questionnaire Form

Swell Surf Check Usability Questionnaire

Name

Please complete the following questionnaire honestly, based on your opinions when using the applications. Thank you for taking part.

Web Application

I found the application easy and intuitive to use

strongly disagree 1 2 3 4 *strongly agree*

I felt that the tasks I was given were easy to carry out

strongly disagree 1 2 3 4 *strongly agree*

The on screen text and images within the application were easy to read and understand

strongly disagree 1 2 3 4 *strongly agree*

It was easy to find the information I wanted in the application

strongly disagree 1 2 3 4 *strongly agree*

I believe that the application was enjoyable to use

strongly disagree 1 2 3 4 *strongly agree*

Android Application

I found the application easy and intuitive to use

strongly disagree 1 2 3 4 *strongly agree*

I felt that the tasks I was given were easy to carry out

strongly disagree 1 2 3 4 *strongly agree*

1 of 2
Created with www.freequestionnairemaker.com

The on screen text and images within the application were easy to read and understand

strongly disagree 1 2 3 4 *strongly agree*

It was easy to find the information I wanted in the application

strongly disagree 1 2 3 4 *strongly agree*

I believe that the application was enjoyable to use

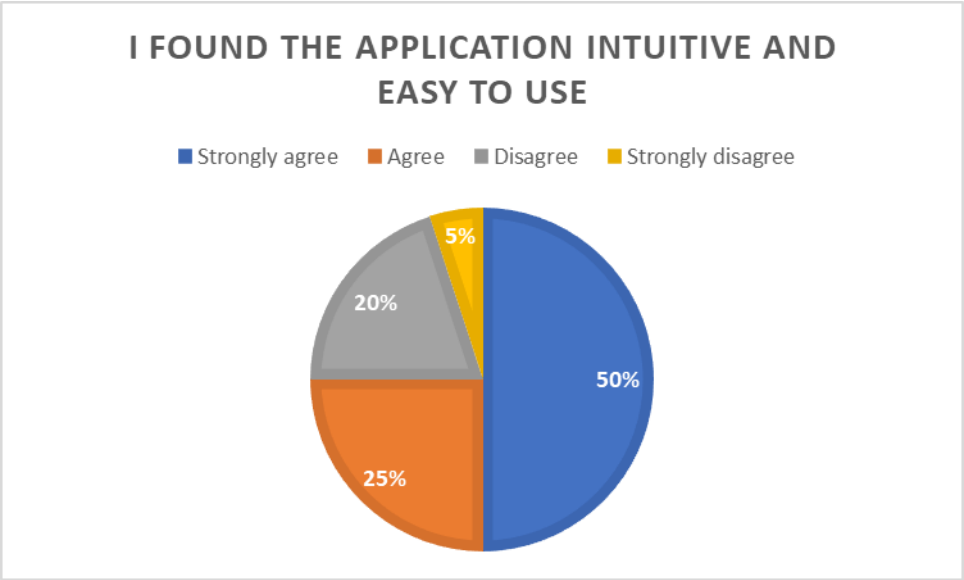
strongly disagree 1 2 3 4 *strongly agree*

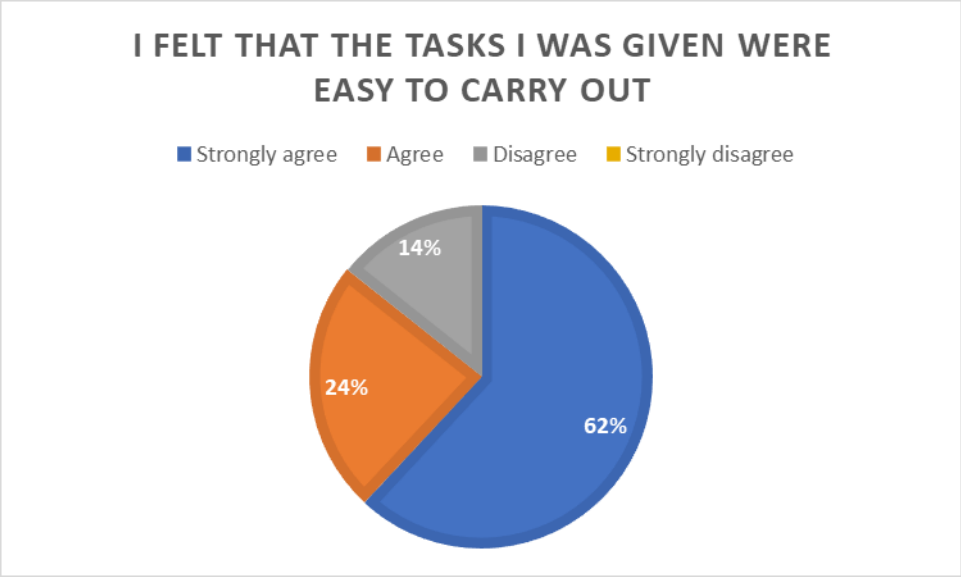
Additional comments

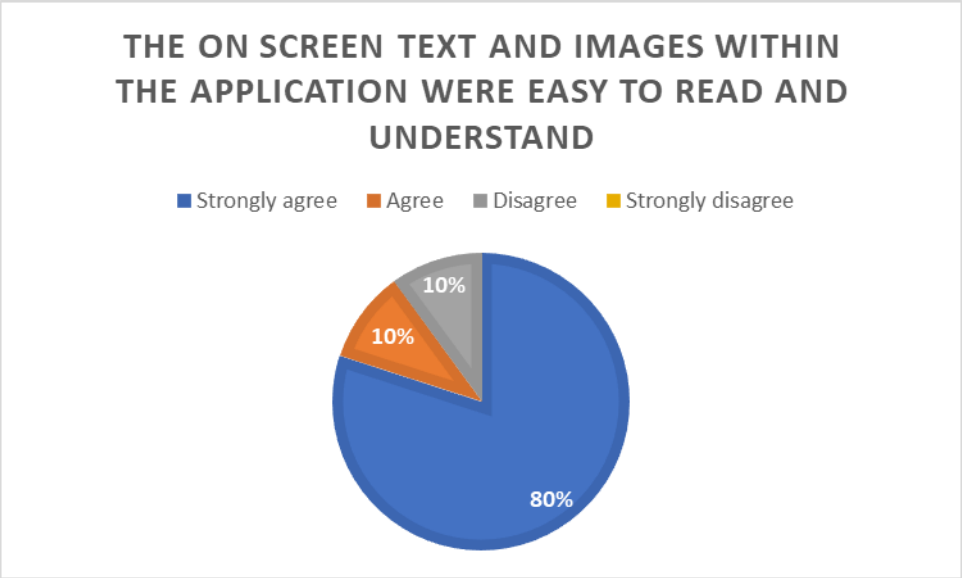
Thank you for taking part in this usability session. Your feedback is valued and appreciated.

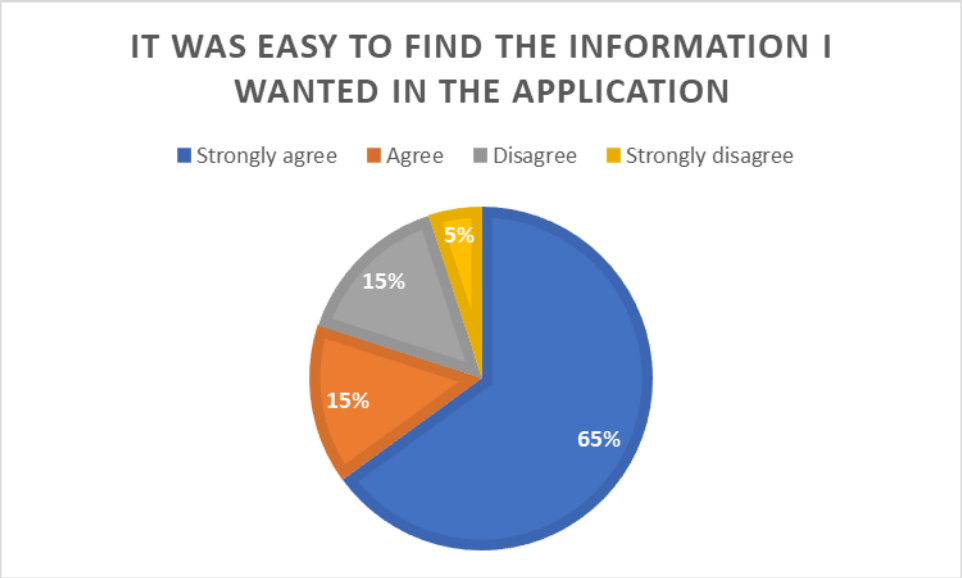
Questionnaire Results

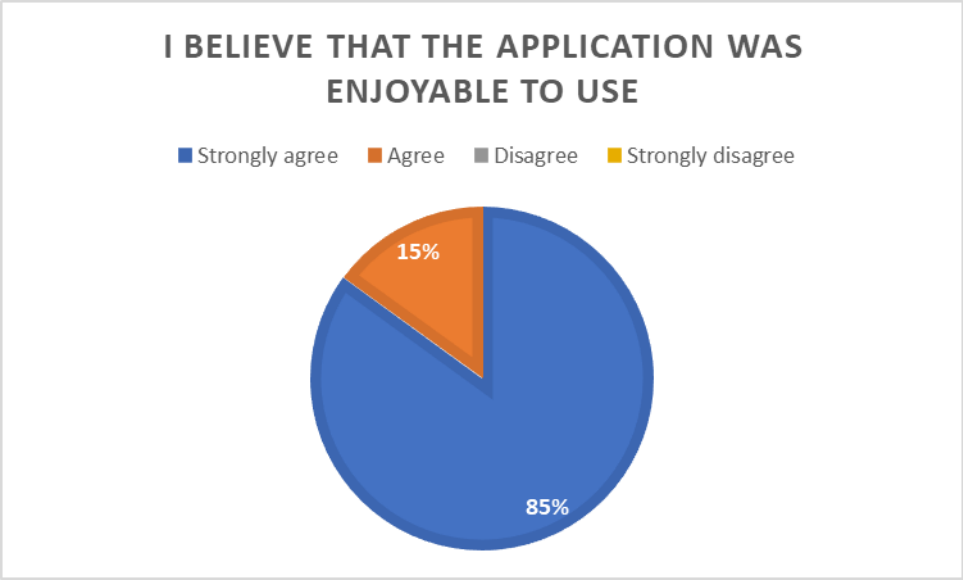
Web Application Round 1



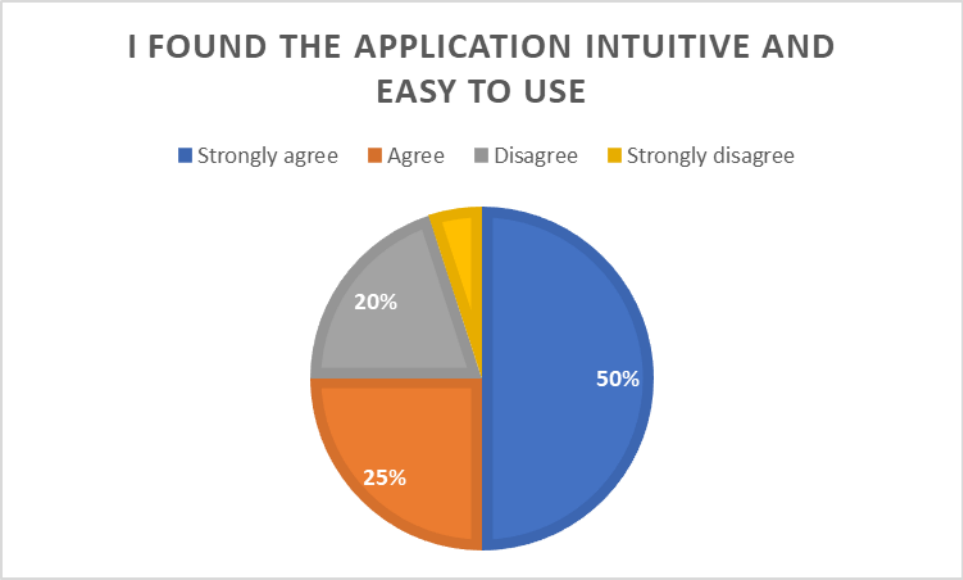

I FELT THAT THE TASKS I WAS GIVEN WERE EASY TO CARRY OUT
Strongly agree
Agree
Disagree
Strongly disagree
14%
24%
62%

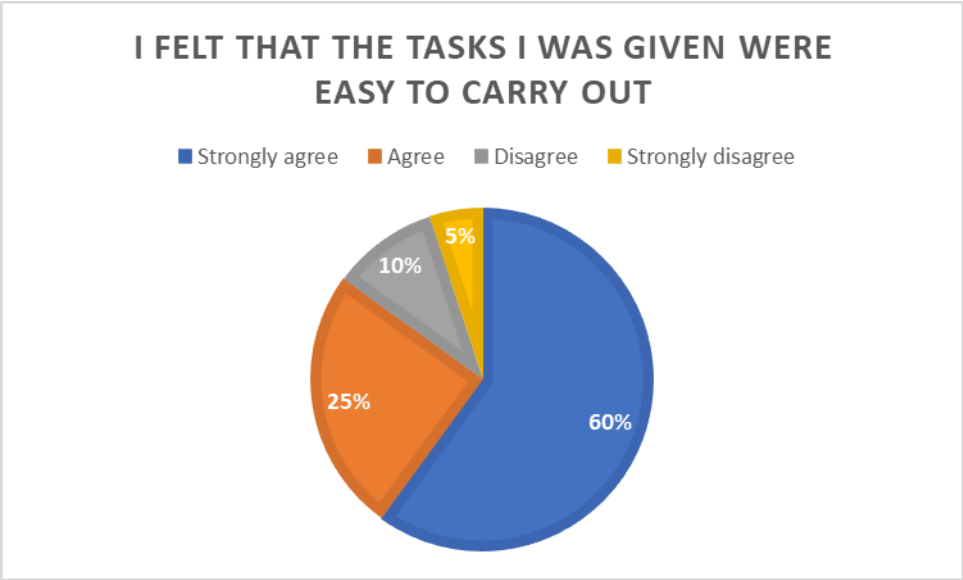

THE ON SCREEN TEXT AND IMAGES WITHIN THE APPLICATION WERE EASY TO READ AND UNDERSTAND
Strongly agree
Agree
Disagree
Strongly disagree
10%
10%
80%

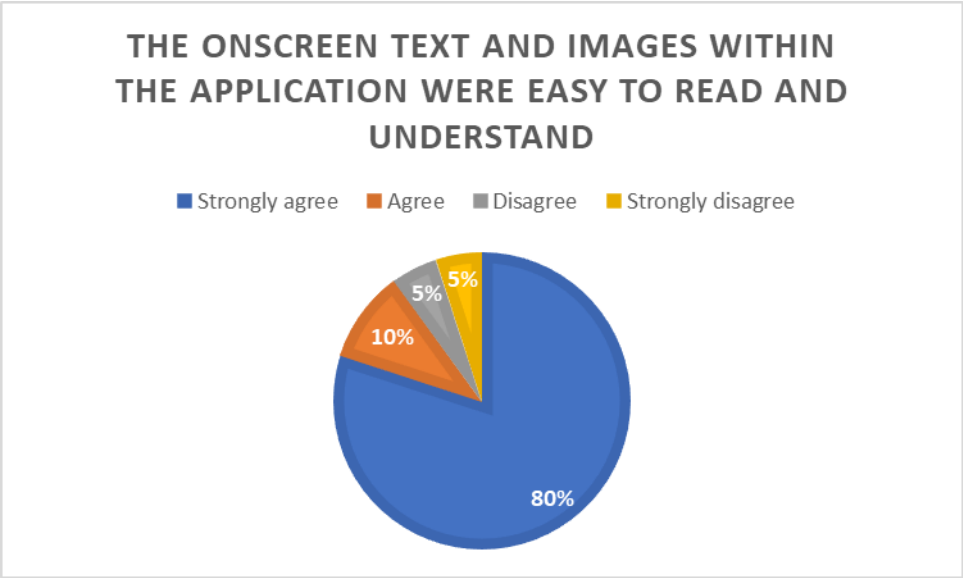

IT WAS EASY TO FIND THE INFORMATION I WANTED IN THE APPLICATION
Strongly agree
Agree
Disagree
Strongly disagree
5%
15%
15%
65%

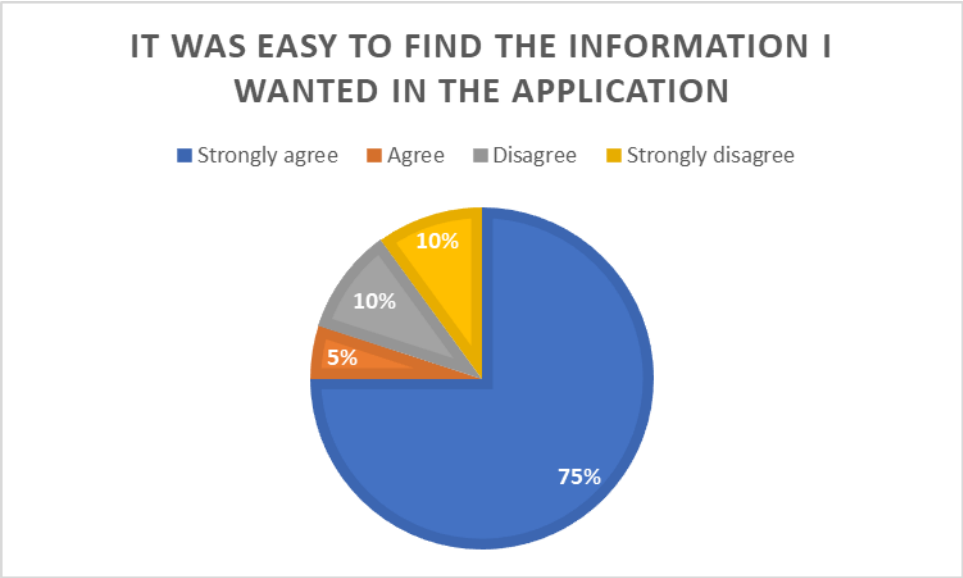


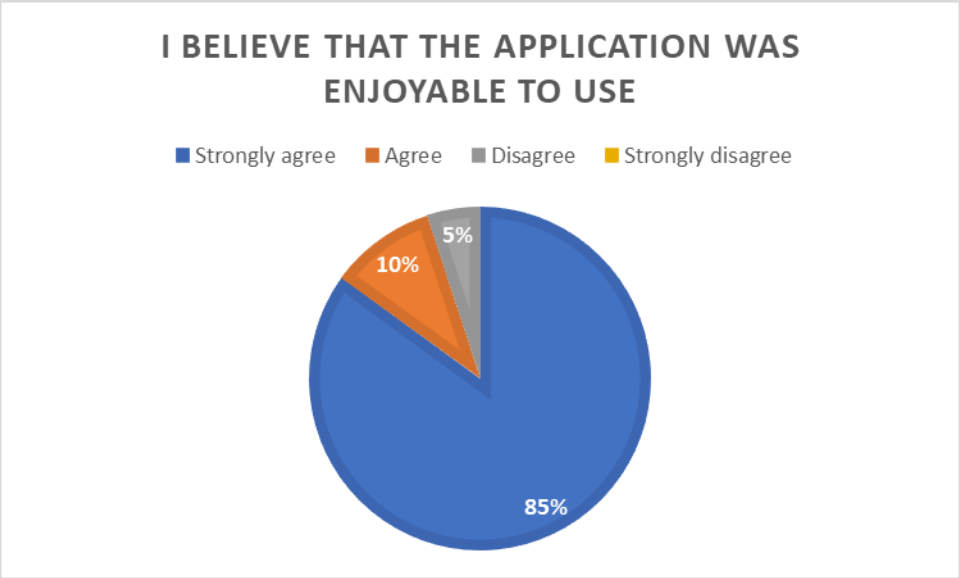
Android Application Round 1



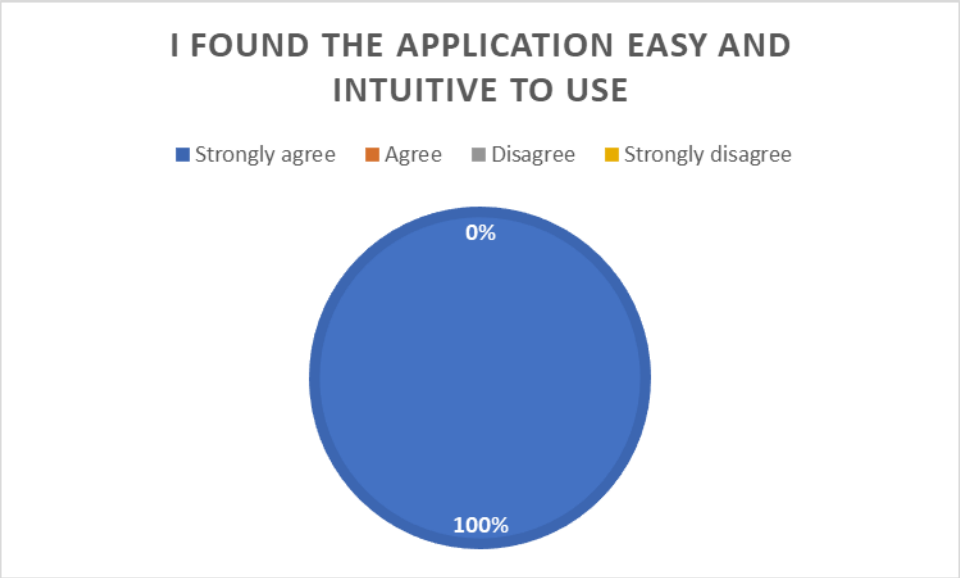

I FELT THAT THE TASKS I WAS GIVEN WERE EASY TO CARRY OUT
Strongly agree
Agree
Disagree
Strongly disagree
5%
10%
25%
60%

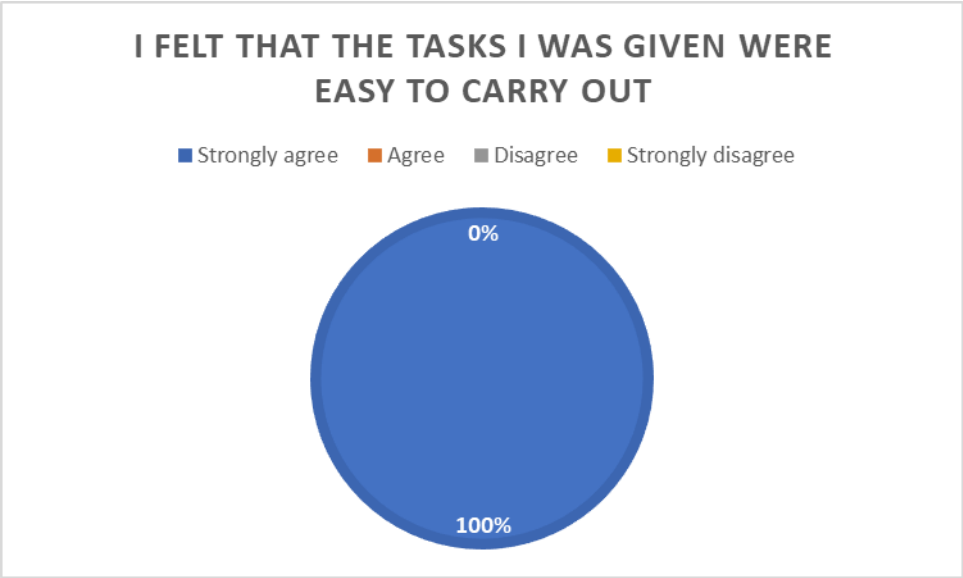

THE ONSCREEN TEXT AND IMAGES WITHIN THE APPLICATION WERE EASY TO READ AND UNDERSTAND
Strongly agree
Agree
Disagree
Strongly disagree
5%
5%
10%
80%

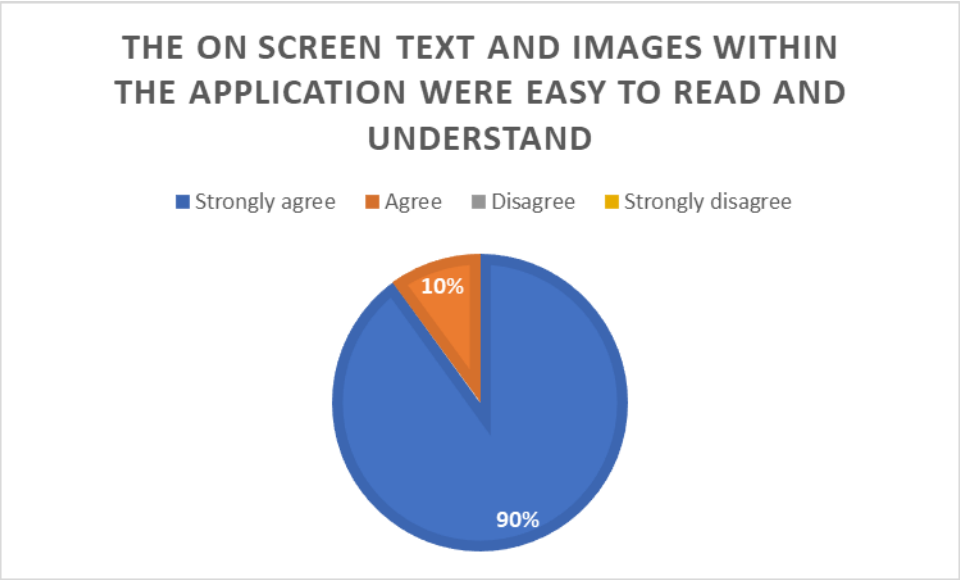

IT WAS EASY TO FIND THE INFORMATION I WANTED IN THE APPLICATION
Strongly agree
Agree
Disagree
Strongly disagree
10%
10%
5%
75%



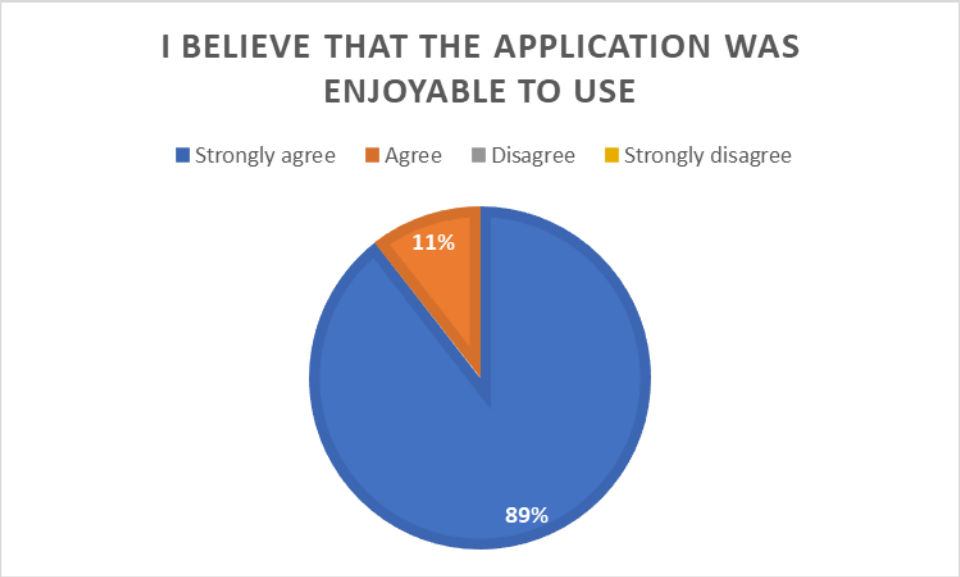
Web Application Round 2



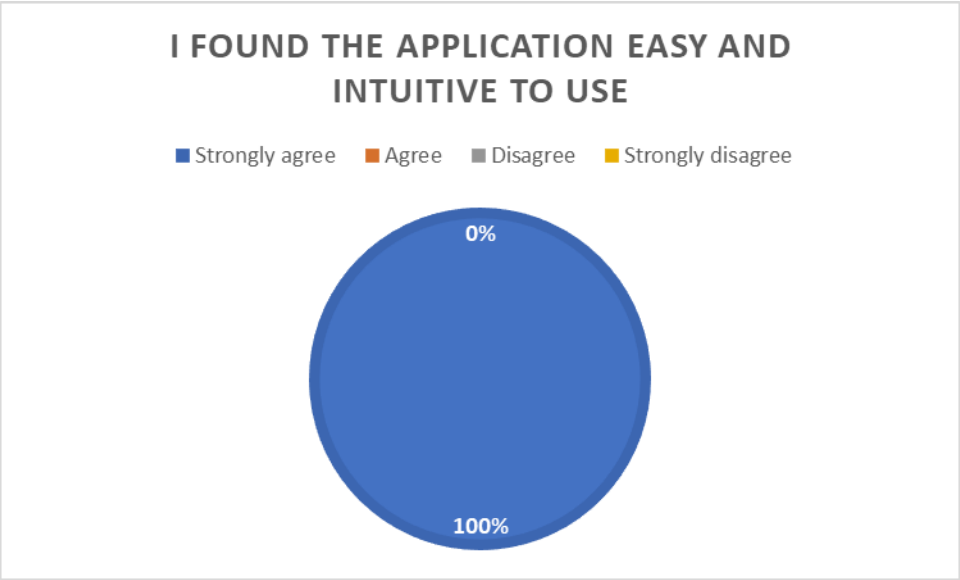

I FELT THAT THE TASKS I WAS GIVEN WERE EASY TO CARRY OUT
Strongly agree
Agree
Disagree
Strongly disagree
0%
100%

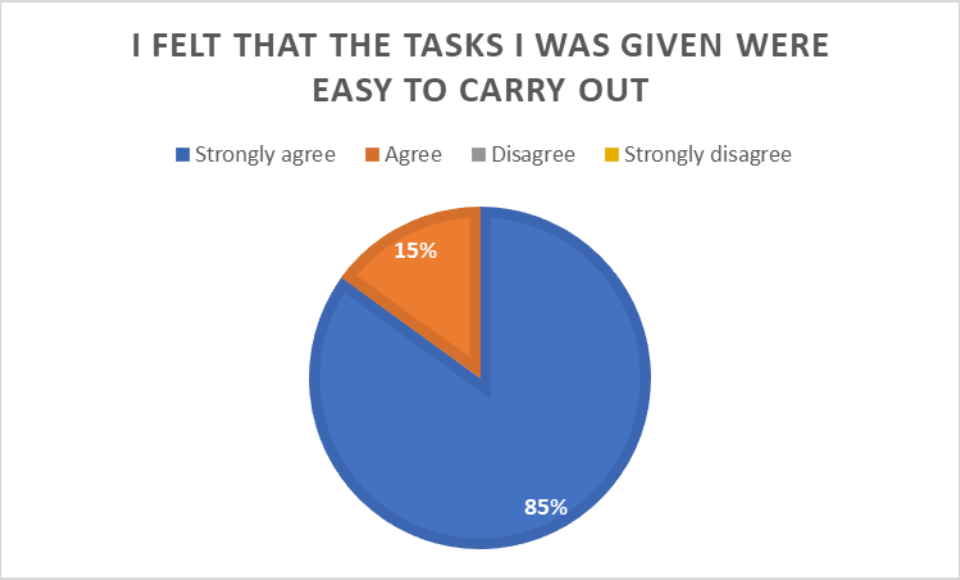

THE ON SCREEN TEXT AND IMAGES WITHIN THE APPLICATION WERE EASY TO READ AND UNDERSTAND
Strongly agree
Agree
Disagree
Strongly disagree
10%
90%

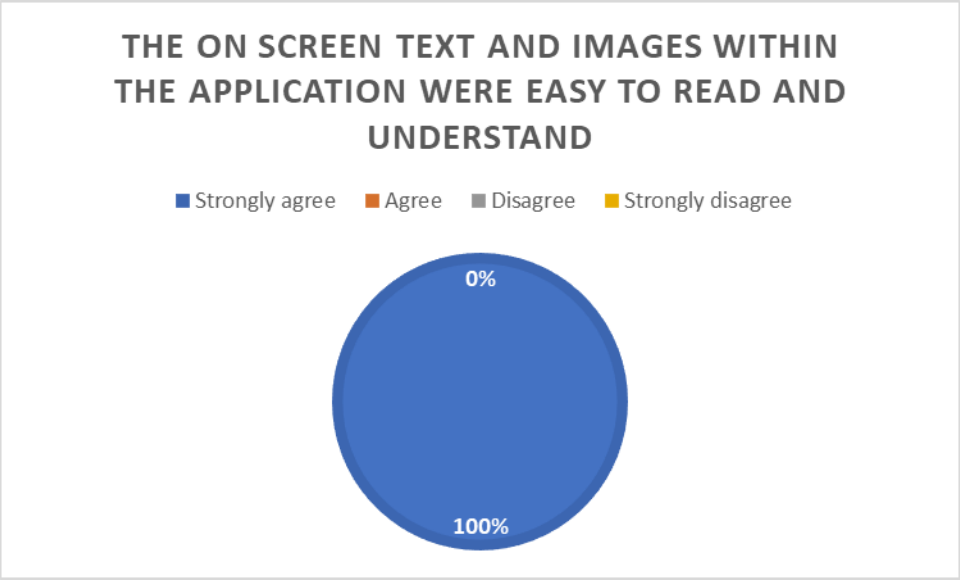
IT WAS EASY TO FIND THE INFORMATION I WANTED IN THE APPLICATION
Strongly agree
Agree
Disagree
Strongly disagree
0%
100%



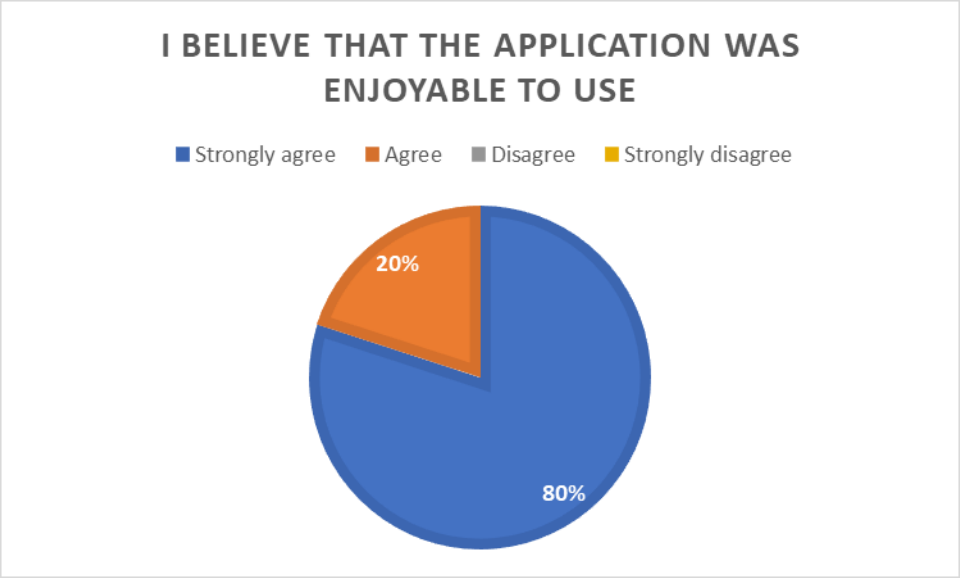
Android Application Round 2




I FELT THAT THE TASKS I WAS GIVEN WERE EASY TO CARRY OUT
Strongly agree
Agree
Disagree
Strongly disagree
15%
85%


THE ON SCREEN TEXT AND IMAGES WITHIN THE APPLICATION WERE EASY TO READ AND UNDERSTAND
Strongly agree
Agree
Disagree
Strongly disagree
0%
100%

IT WAS EASY TO FIND THE INFORMATION I WANTED IN THE APPLICATION
Strongly agree
Agree
Disagree
Strongly disagree
0%
100%



15.7 API Endpoints

default

GET	/swellregions
OPTIONS	/swellregions
GET	/swellspots
OPTIONS	/swellspots
GET	/swellspots/region
OPTIONS	/swellspots/region
GET	/swellspots/spot
OPTIONS	/swellspots/spot
GET	/swellspots/spotname
OPTIONS	/swellspots/spotname
GET	/swellusers
POST	/swellusers
OPTIONS	/swellusers
POST	/swellusers/favourite
OPTIONS	/swellusers/favourite
DELETE	/swellusers/favourite/remove
OPTIONS	/swellusers/favourite/remove
GET	/swellusers/user
OPTIONS	/swellusers/user

15.8 AWS Lambda Code

Get spots by name

```
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "spots";
  var hasError = false;
  var documents = [];
  const name = event.queryStringParameters.name

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({ spotName: name });
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }
```

```
    if (err !== null) {
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }


  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

Get spot for region

```
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "spots";
  var hasError = false;
  var documents = [];
  const id = event.queryStringParameters.id

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
```

```
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({ surflineRegionId: id });
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }

    if (err !== null) {
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }


  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

Get regions

```
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "regions";
  var hasError = false;
  var documents = [];

  updateEventLoopIgnore();
```

```
  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({});
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }

    if (err !== null) {
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }

  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

Get user

```
var ObjectID = require('mongodb').ObjectID;

var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "users";
  var hasError = false;
  var documents = [];
  var username = event.queryStringParameters.username;

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({ username: username });
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }

    if (err !== null) {
```

```
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }


  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

Register user

```
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "users";
  var hasError = false;
  var jsonContents = JSON.parse(event.body);

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
```

```
  }

  function onConnected(db) {
    createDoc(db, jsonContents, callback);
  }

  function createDoc (db, json, callback) {
    db.collection(collectionName).insertOne( json, function(err, result) {
        if(err!=null) {
            console.error("an error occurred in createDoc", err);
            callback(null, JSON.stringify(err));
        }
        else {
          console.log("Successfully registered user: " + result.insertedId);
          callback(null, { statusCode: 200, headers: {"Access-Control-Allow-
Origin": '*'}, body: JSON.stringify(jsonContents) });
        }
    });
  }
}
```

Compare password

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

exports.handler = (event, context, callback) => {

var candidatePw = event.queryStringParameters.candidatePw;
var userPw = event.queryStringParameters.userPw;

//var candidatePw = "qwerty";
var res = {
    success: true
};

bcrypt.compare(candidatePw, userPw, (err, isMatch) => {
    //if (err) throw err;
    if (isMatch) {
        console.log("match");
        const token = jwt.sign({}, "secret", {
            expiresIn: 604800 // 1 week
        });

        var res = {
            success: true,
            token: 'JWT ' + token
        }

        callback(null, { statusCode: 200, headers: {"Access-Control-Allow-
Origin": '*'}, body: JSON.stringify(res) });

    } else {
        console.log("no match");
        var res = {
```

```
            success: false
        }

        callback(null, { statusCode: 200, headers: {"Access-Control-Allow-
Origin": '*'}, body: JSON.stringify(res) });
    }
});
}
```

Add spot to favourites

```
var ObjectID = require('mongodb').ObjectID;
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "users";
  var hasError = false;
  var jsonContents = JSON.parse(event.body);
  var o_id = new ObjectID(event.queryStringParameters.userId);
  // var o_id = new ObjectID("5df40e041c9d44000017f460");

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
  }

  function onConnected(db) {
    insertSpot(db, jsonContents, callback);
  }

  function insertSpot (db, json, callback) {
    db.collection(collectionName).update({_id: o_id}, { $addToSet: {
favourites: json } }, function(err, res) {
```

```
      if(err!=null) {
        console.error("an error occurred", err);
        callback(null, JSON.stringify(err));
      }
      else {
        console.log("Successfully added spot to favourites");
        callback(null, { statusCode: 200, headers: {"Access-Control-Allow-
Origin": '*'}, body: JSON.stringify(jsonContents) });
      }
    });
  }
}
```

Delete spot from favourites

```
var ObjectID = require('mongodb').ObjectID;
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "users";
  var hasError = false;
  var o_id = new ObjectID(event.queryStringParameters.id);
  var spot = event.queryStringParameters.spot;
  //var o_id = new ObjectID("5e0a3cd7cc44a3e33aa28499");
  //var spot = "5e0a33951c9d4400008fbab8";


  console.log("event spot " + spot);

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
```

```
  }

  function onConnected(db) {
    removeSpot(db, spot, callback);
  }

  function removeSpot (db, spot, callback) {
    db.collection(collectionName).update({_id: o_id}, { $pull: { favourites:
{spot: spot} } }, function(err, res) {
      if(err!=null) {
        console.error("an error occurred", err);
        callback(null, JSON.stringify(err));
      }
      else {
        console.log("Successfully added spot to favourites");
        callback(null, { statusCode: 200, headers: {"Access-Control-Allow-
Origin": '*'}, body: JSON.stringify(spot) });
      }
    });
  }
}
```

Get spots

```
var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "spots";
  var hasError = false;
  var documents = [];

  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
```

```
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({});
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }

    if (err !== null) {
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }

  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

Get spot by ID

```
var ObjectID = require('mongodb').ObjectID;

var MongoClient = require("mongodb").MongoClient;

exports.handler = (event, context, callback) => {
  var connectionString =
    "mongodb://emilypotter:N3llytheelephan!@cluster0-shard-00-00-
8ddti.mongodb.net:27017,cluster0-shard-00-01-8ddti.mongodb.net:27017,cluster0-
shard-00-02-8ddti.mongodb.net:27017/swelldb?ssl=true&replicaSet=Cluster0-
shard-0&authSource=admin&retryWrites=true";
  var collectionName = "spots";
  var hasError = false;
  var documents = [];
  var o_id = new ObjectID(event.queryStringParameters.id);
```

```
  updateEventLoopIgnore();

  connect(connectionString);

  function updateEventLoopIgnore() {
    var ignoreEventLoop = process.env["IGNORE_EVENT_LOOP"];
    if (ignoreEventLoop == "YES") {
      context.callbackWaitsForEmptyEventLoop = false;
    }
  }

  function connect(connectionString) {
    MongoClient.connect(connectionString, onCallCompleted);
  }

  function onCallCompleted(err, db) {
    if (err !== null) {
      console.log(err);
      return;
    }
    onConnected(db);
  }

  function onConnected(db) {
    var cursor = filterFunction(db.collection(collectionName));
    cursor.each(onProcessCursorItem);
  }

  function filterFunction(collection) {
    return collection.find({ _id: o_id });
  }

  function onProcessCursorItem(err, document) {
    if (hasError) {
      return;
    }

    if (err !== null) {
      hasError = true;
      console.log("cursor error: " + err);
    }

    if (document === null) {
      sendDocuments();
      return;
    }

    documents.push(document);
  }


  function sendDocuments() {
    callback(null, { statusCode: 200, headers: {"Access-Control-Allow-Origin":
'*'}, body: JSON.stringify(documents) });
  }
};
```

15.9 Dialogflow Fulfilment Code

```
'use strict';

const functions = require('firebase-functions');
const {google} = require('googleapis');
const {WebhookClient} = require('dialogflow-fulfillment');
const https = require('https');
var rp = require('request-promise-native');

process.env.DEBUG = 'dialogflow:*'; // It enables lib debugging statements

exports.dialogflowFirebaseFulfillment = functions.https.onRequest((request,
response) => {
  const agent = new WebhookClient({ request, response });

  function getResponse (agent) {
    //mongo request
    let spotName = agent.parameters['geo-city'];
    console.log(agent.parameters['geo-city']);
    var url = `https://tetqc1kgx7.execute-api.eu-west-
2.amazonaws.com/prod/swellspots/spotname?name=${spotName}`;
      var options = {
      uri: url,
      json: true
      };
      return rp.get( options )
    .then( body => {
      if (body[0] != undefined) {
          let id = body[0].surflineLongId;
        console.log(id);

      //surfline request
      var url =
`https://services.surfline.com/kbyg/spots/forecasts/conditions?spotId=${id}&da
ys=1`;
            var options = {
          uri: url,
          json: true
        };
            return rp.get( options )
      .then( body => {
            console.log(body);
            let conditions = body.data.conditions[0].observation;
            agent.add(conditions);
            });
        } else {
          agent.add("Sorry, I was unable to get the conditions for you. Either
a spot wasn't specified or I don't yet know the spot you are looking for.");
        }
    });
  }
```

```
  let intentMap = new Map();
  intentMap.set('Check Surf', getResponse);
  agent.handleRequest(intentMap);
});
```